# Transformers, LLM et LoRA

Formation complète et questions d'entretien : de l'attention au fine-tuning avec LoRA et QLoRA, jusqu'au déploiement.

- **Partie A** : Formation Transformers, LLM et LoRA (24 modules : attention, inférence, LoRA, QLoRA, alignement, déploiement)
- **Partie B** : Questions d'entretien Transformers, LLM et LoRA (plus de 200 questions, conception de systèmes, code)

Généré le 12/10/2026. Source : dossier `docs/` du dépôt.

# Sommaire


## Partie A : Formation Transformers, LLM et LoRA

Sommaire
1. Vue d'ensemble : du texte à la réponse
2. La tokenisation
3. Embeddings et positions
4. Le mécanisme d'attention
5. Le bloc Transformer et l'architecture complète
6. Compter les paramètres et le calcul
7. Le pré-entraînement
8. La génération et l'inférence
9. La mémoire GPU et l'entraînement à grande échelle
10. Prompting, RAG ou fine-tuning : choisir
11. Fine-tuning complet et méthodes PEFT
12. LoRA en profondeur
13. QLoRA et les variantes de LoRA
14. Fine-tuner en pratique avec Hugging Face
15. Les données
16. L'alignement : SFT, RLHF, DPO, GRPO
17. Évaluer un modèle fine-tuné
18. Quantification, fusion et déploiement
19. Architectures modernes : MoE, long contexte, multimodal, raisonnement
20. Sécurité, limites et licences
21. Labs et exercices
22. Plan de révision sur trois semaines
23. Aide-mémoire : formules et ordres de grandeur
24. Glossaire

## Partie B : Questions d'entretien Transformers, LLM et LoRA

Sommaire
A. LLM et tokenisation
B. Attention et Transformer
C. Pré-entraînement
D. Inférence et KV cache
E. Mémoire et entraînement distribué
F. Choisir et fine-tuning
G. LoRA
H. QLoRA et variantes
I. Données, alignement et évaluation
J. Quantification et déploiement
K. Conception de systèmes
L. Questions pièges
M. Exercices de code
N. Les erreurs à éviter

PARTIE A

# Formation Transformers, LLM et LoRA

# Formation Transformers, LLM et LoRA de A à Z

Un cours complet pour comprendre **comment fonctionne un LLM** (modèle de langage de grande taille), **comment on l'entraîne, l'adapte et le déploie**, et surtout **LoRA** et ses variantes (QLoRA, DoRA...). Il complète la formation RAG : le RAG apporte des connaissances au moment de la requête, le fine-tuning (ici LoRA) change le comportement du modèle.

Les questions d'entretien (plus de 200, avec réponses) sont dans `entretien-transformers-lora.md`. Le code du cours est dans un fichier exécutable : `labs/mini_transformer_lora.py`.

Comment lire ce document :

- **Partie I** : le Transformer et les LLM, de la tokenisation à l'inférence.
- **Partie II** : adapter un LLM, avec LoRA au centre.
- **Partie III** : aller plus loin, labs, plan de révision, glossaire, aide-mémoire.
- Les extraits de code marqués **« vérifié »** ont été exécutés (fichier `labs/mini_transformer_lora.py`, PyTorch, sur CPU). Les extraits Hugging Face (`transformers`, `peft`, `trl`) sont marqués **« non exécuté ici »** : les noms d'arguments changent d'une version à l'autre, vérifie la documentation de la version installée.
- Les chiffres venant de calculs sont donnés avec leur formule. Les « repères » (valeurs typiques de la pratique) sont des points de départ, pas des lois : **mesure sur tes données**.

## Sommaire

**Partie I : les fondations**

1. Vue d'ensemble : du texte à la réponse
2. La tokenisation
3. Embeddings et positions
4. Le mécanisme d'attention
5. Le bloc Transformer et l'architecture complète
6. Compter les paramètres et le calcul
7. Le pré-entraînement
8. La génération et l'inférence
9. La mémoire GPU et l'entraînement à grande échelle

**Partie II : adapter un LLM**
10. Prompting, RAG ou fine-tuning : choisir 11. Fine-tuning complet et méthodes PEFT 12. LoRA en profondeur 13. QLoRA et les variantes de LoRA 14. Fine-tuner en pratique avec Hugging Face 15. Les données 16. L'alignement : SFT, RLHF, DPO, GRPO 17. Évaluer un modèle fine-tuné 18. Quantification, fusion et déploiement

**Partie III : aller plus loin**

19. Architectures modernes : MoE, long contexte, multimodal, raisonnement 20. Sécurité, limites et licences 21. Labs et exercices 22. Plan de révision sur trois semaines 23. Aide-mémoire : formules et ordres de grandeur 24. Glossaire

---

# PARTIE I : LES FONDATIONS

## 1. Vue d'ensemble : du texte à la réponse

### Ce qu'est un LLM

Un LLM est un réseau de neurones qui, étant donné une suite de **tokens** (morceaux de texte), prédit la **distribution de probabilité du token suivant**. C'est tout. Tout le reste (répondre à des questions, écrire du code, résumer) découle de cette prédiction répétée.

```
"Le chat dort sur le"
        |  1. tokenisation : texte -> identifiants entiers          [ 791, 8415, 34414, 389, 279 ]
        v
        |  2. embeddings : chaque id -> un vecteur de dimension d    (T x d)
        v
        |  3. N blocs Transformer (attention + réseau feed-forward)  (T x d)
        v
        |  4. projection finale : vecteur -> un score par token du vocabulaire   (T x V) : les logits
        v
        |  5. softmax : scores -> probabilités   canapé 0,31 | lit 0,22 | sol 0,09 | ...
        v
        |  6. choix du token (glouton ou échantillonnage) : « canapé »
        v
   on ajoute le token à l'entrée et on recommence (génération autorégressive)
```

### L'idée de l'autorégression

La probabilité d'une phrase se décompose en produit de probabilités conditionnelles :

```
P(t1, t2, ..., tn) = P(t1) × P(t2 | t1) × P(t3 | t1, t2) × ... × P(tn | t1..t(n-1))
```

Le modèle apprend à estimer chaque facteur. Générer = tirer un token, l'ajouter, recommencer.

### Pourquoi le Transformer a gagné

Avant 2017, on utilisait des **réseaux récurrents** (RNN, LSTM) qui lisent le texte mot par mot. Deux problèmes : (1) calcul **séquentiel**, impossible à paralléliser sur GPU ; (2) l'information du début s'estompe sur les longues séquences. Le Transformer remplace la récurrence par l'**attention** : chaque token peut regarder **directement** tous les autres, et tout se calcule en **parallèle**. C'est ce qui a permis de passer à l'échelle.

### Repères historiques (à connaître pour situer les idées)

| Année | Événement | Pourquoi c'est important |
|---|---|---|
| 2013 | word2vec | Les mots deviennent des vecteurs |

| Année | Événement | Pourquoi c'est important |
|---|---|---|
| 2014 | Attention pour la traduction (Bahdanau) | Premier mécanisme d'attention, sur un RNN |
| 2017 | « Attention Is All You Need » (Vaswani et al.) | Le Transformer : plus de récurrence |
| 2018 | GPT et BERT | Pré-entraînement puis adaptation ; deux familles (decoder / encoder) |
| 2019-2020 | GPT-2, GPT-3, T5, lois d'échelle (Kaplan) | Plus gros = mieux, de façon prévisible ; apprentissage en contexte |
| 2021 | **LoRA** (Hu et al.) | Adapter un gros modèle en entraînant très peu de paramètres |
| 2022 | InstructGPT (RLHF), Chinchilla | Aligner un modèle ; équilibrer taille du modèle et quantité de données |
| 2023 | Llama, **QLoRA**, DPO, FlashAttention-2 | Modèles ouverts ; fine-tuning sur un seul GPU ; alignement simplifié |
| 2024-2025 | Modèles de raisonnement, GRPO, MoE ouverts | Entraînement par renforcement sur récompenses vérifiables |

### Les trois grandes phases de la vie d'un LLM

1. **Pré-entraînement** : prédire le token suivant sur des milliers de milliards de tokens. Coûte des millions de dollars. Donne un « modèle de base » qui complète du texte.
2. **Post-entraînement** : SFT (apprendre à suivre des instructions), puis alignement sur des préférences (RLHF, DPO). Donne un modèle « instruct » ou « chat ».
3. **Adaptation et déploiement** : fine-tuning léger (LoRA) pour un cas d'usage, quantification, serving. **C'est le périmètre d'un ingénieur** : rarement le pré-entraînement.

---

## 2. La tokenisation

### Pourquoi des tokens

Le modèle ne lit pas des lettres ni des mots, mais des **identifiants entiers** d'un vocabulaire fixe (typiquement 32 000 à 250 000 entrées). Un token est un morceau de mot fréquent.

- **Mots entiers** : vocabulaire énorme, mots inconnus impossibles à traiter.
- **Caractères** : vocabulaire minuscule, mais séquences très longues (attention coûteuse).
- **Sous-mots** (le compromis retenu) : les mots fréquents sont un seul token, les mots rares sont découpés en morceaux. Aucun mot n'est « inconnu ».

### Les algorithmes

| Algorithme | Principe | Utilisé par |
|---|---|---|
| **BPE** (*Byte Pair Encoding*) | On part de petites unités et on fusionne **la paire la plus fréquente**, en boucle | GPT-2/3/4, Llama 3, nombreux autres |

| Algorithme | Principe | Utilisé par |
|---|---|---|
| **BPE au niveau des octets** | Même chose, mais l'alphabet de départ est les 256 octets : **aucun caractère inconnu** | GPT-2 et successeurs |
| **WordPiece** | Comme BPE, mais choisit les fusions qui maximisent la vraisemblance | BERT |
| **Unigram / SentencePiece** | Part d'un gros vocabulaire et **retire** les tokens peu utiles | T5, Llama 1-2 (SentencePiece) |

## BPE : un algorithme à savoir écrire (vérifié)

```
from collections import Counter

def merge_pair(tokens, pair, new_id):
    out, i = [], 0
    while i < len(tokens):
        if i < len(tokens) - 1 and (tokens[i], tokens[i + 1]) == pair:
            out.append(new_id); i += 2
        else:
            out.append(tokens[i]); i += 1
    return out

def train_bpe(text, num_merges):
    tokens = list(text.encode("utf-8"))            # on part des 256 octets
    merges = {}
    for i in range(num_merges):
        pairs = Counter(zip(tokens, tokens[1:]))   # fréquence de chaque paire voisine
        if not pairs:
            break
        best = max(pairs, key=pairs.get)           # la paire la plus fréquente
        merges[best] = 256 + i                     # nouveau token
        tokens = merge_pair(tokens, best, 256 + i)
    return merges

def bpe_encode(text, merges):
    tokens = list(text.encode("utf-8"))
    for pair, new_id in merges.items():            # dans l'ordre d'apprentissage
        tokens = merge_pair(tokens, pair, new_id)
    return tokens

def bpe_decode(ids, merges):
    vocab = {i: bytes([i]) for i in range(256)}
    for (a, b), new_id in merges.items():
        vocab[new_id] = vocab[a] + vocab[b]
    return b"".join(vocab[i] for i in ids).decode("utf-8", errors="replace")
```

Propriétés vérifiées : `decode(encode(texte)) == texte` (avec accents et emoji), et le texte encodé a moins de tokens que d'octets. **L'encodage est réversible et ne produit jamais « inconnu ».**

### Conséquences pratiques (questions fréquentes)

- **Les langues ne sont pas égales** : un vocabulaire entraîné surtout sur de l'anglais découpe le français, l'arabe ou le japonais en **plus** de tokens. Même texte = plus de tokens = plus cher et moins de contexte utile.
- **Les nombres et le code** : « 1234 » peut devenir « 12 » + « 34 » ; l'arithmétique et l'orthographe (compter les lettres de « fraise ») sont difficiles car le modèle ne voit pas les lettres.
- **Les espaces comptent** : « chat » et « ␣chat » sont deux tokens différents.
- **Tokens spéciaux** : début de séquence ( `<s>` ), fin ( `</s>` ou `<|endoftext|>` ), séparateurs de rôles dans un chat. **Utiliser les tokens spéciaux du modèle** : une erreur ici dégrade silencieusement le fine-tuning (voir module 14).
- **Un tokenizer est lié à son modèle** : on ne le change pas sans ré-entraîner l'embedding.
- Règle d'estimation : environ 3/4 de mot par token en anglais ; **mesure** sur ton texte.

---

## 3. Embeddings et positions

### La table d'embeddings

Chaque token id est transformé en un vecteur de dimension `d` (appelée `d_model` : 768, 4096, 8192...) par une simple **table de correspondance** : une matrice `E` de taille `V × d`. Ces vecteurs sont **appris** pendant l'entraînement.

- Sortie du modèle : le vecteur final est projeté vers `V` scores (« logits ») par une matrice de taille `d × V`. Souvent on **réutilise la même matrice** (*weight tying*) pour l'entrée et la sortie : cela économise `V × d` paramètres. Notre mini-modèle le fait.
- Coût en paramètres : `V × d` (Llama-2 7B : 32 000 × 4 096 = 131 M, pour l'entrée, autant pour la sortie car non partagées).

### Le problème de l'ordre

L'attention est **invariante à l'ordre** : sans information de position, « le chien mord l'homme » et « l'homme mord le chien » donnent les mêmes calculs. Il faut **injecter la position**.

| Méthode | Principe | Utilisée par |
|---|---|---|
| **Sinusoïdale** | On **ajoute** un vecteur calculé par des sinus/cosinus de fréquences variées | Transformer original |
| **Apprise** | Une table `T_max × d` apprise, ajoutée à l'embedding | GPT-2, BERT |
| **RoPE** (*Rotary*) | On **fait tourner** les vecteurs `q` et `k` d'un angle proportionnel à la position | Llama, Mistral, Qwen, la plupart des modèles récents |
| **ALiBi** | On ajoute aux scores d'attention un **malus proportionnel à la distance** | MPT, BLOOM |

### RoPE : l'idée à savoir expliquer

On découpe chaque vecteur `q` ou `k` en **paires de dimensions** et on fait tourner chaque paire d'un angle `position × fréquence` (fréquences différentes selon la paire). Résultat mathématique clé : le produit scalaire `q·k` après rotation **ne dépend que de l'écart de position** `m − n`, pas des positions absolues. L'attention capte donc naturellement la **position relative**.

```
def rope(x, positions, base=10000.0):              # vérifié
    d = x.size(-1)
    freqs = base ** (-torch.arange(0, d, 2, dtype=torch.float32) / d)
    angles = positions[:, None].float() * freqs[None, :]
    cos, sin = angles.cos(), angles.sin()
    x1, x2 = x[..., 0::2], x[..., 1::2]
    out = torch.empty_like(x)
    out[..., 0::2] = x1 * cos - x2 * sin
    out[..., 1::2] = x1 * sin + x2 * cos
    return out
```

Test vérifié : `rope(q, 5) · rope(k, 2)` est égal à `rope(q, 40) · rope(k, 37)` (même écart de 3), et différent de `rope(q, 40) · rope(k, 30)` (écart de 10).

### Étendre le contexte

Un modèle entraîné sur 4 096 positions se dégrade au-delà. Les techniques d'extension (**interpolation de positions**, NTK-aware, YaRN) **compressent** les angles RoPE pour que les positions longues restent dans la plage vue à l'entraînement, souvent avec un court fine-tuning. Voir module 19.

---

## 4. Le mécanisme d'attention

### L'intuition : une recherche souple dans un dictionnaire

Pour chaque token, le modèle se demande : « parmi les autres tokens, lesquels sont pertinents pour moi, et quelle information en récupérer ? ». Chaque token produit trois vecteurs, par trois projections linéaires apprises :

- **Query** `q` : « ce que je cherche » ;
- **Key** `k` : « ce que je contiens, pour être trouvé » ;
- **Value** `v` : « l'information que je transmets si on me choisit ».

On compare la query à toutes les keys (produit scalaire), on convertit en poids avec un softmax, et on prend la moyenne pondérée des values.

### La formule (à connaître par cœur)

$$\text{Attention}(Q, K, V) = \text{softmax}\left( Q K^{\intercal} / \sqrt{d_k} \right) V$$

Avec `T` tokens et une dimension de tête `d_k` :

- `Q`, `K`, `V` : matrices `T × d_k` ;
- `Q Kᵀ` : matrice `T × T` des scores (qui regarde qui) ;
- on divise par `√d_k` ; le softmax est appliqué **par ligne** ; le résultat `T × d_k`.

**Pourquoi diviser par `√d_k` ?** Si les composantes de `q` et `k` sont de variance 1, leur produit scalaire a une variance de `d_k`. Sans normalisation, les scores deviennent grands, le softmax sature (une probabilité proche de 1, les autres proches de 0) et les **gradients s'annulent**. Diviser par `√d_k` ramène la variance à 1.

## Le masque causal (modèles « decoder »)

Pour prédire le token suivant, un token **ne doit pas voir le futur**. On met `−∞` dans les scores au-dessus de la diagonale avant le softmax : leur poids devient exactement 0.

```
       k0  k1  k2  k3
  q0 [  ok  -∞  -∞  -∞ ]
  q1 [  ok  ok  -∞  -∞ ]
  q2 [  ok  ok  ok  -∞ ]
  q3 [  ok  ok  ok  ok ]
```

Grâce à ce masque, on entraîne sur **toutes les positions d'une séquence en un seul passage** (chaque position prédit son suivant) : c'est ce qui rend l'entraînement efficace.

## Multi-têtes

On découpe `d_model` en `h` têtes de dimension `d_k = d_model / h`. Chaque tête a ses propres projections et calcule son attention en parallèle ; les sorties sont concaténées puis projetées par `W_O`. Intérêt : chaque tête peut se spécialiser dans un type de relation (syntaxe, coréférence, position...).

## Le code complet (vérifié)

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.n_heads, self.head_dim = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=False)   # Q, K, V en un seul produit
        self.proj = nn.Linear(d_model, d_model, bias=False)       # W_O

    def forward(self, x, past=None):
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)
        heads = lambda t: t.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)
        q, k, v = heads(q), heads(k), heads(v)                    # (B, h, T, head_dim)
        if past is not None:                                      # KV cache (module 8)
            k = torch.cat([past[0], k], dim=2)
            v = torch.cat([past[1], v], dim=2)
        S = k.size(2)
```

```
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        mask = torch.ones(T, S, dtype=torch.bool, device=x.device).tril(diagonal=S - T)
        scores = scores.masked_fill(~mask, float("-inf"))
        out = (scores.softmax(-1) @ v).transpose(1, 2).reshape(B, T, C)
        return self.proj(out), (k, v)
```

Propriété vérifiée : si on **change un token futur**, les sorties des positions précédentes ne changent pas du tout (causalité).

### Le coût : le problème quadratique

La matrice de scores est `T × T` par tête et par couche.

- **Temps** : `O(T² · d)`. Doubler le contexte = environ quatre fois plus de calcul d'attention.
- **Mémoire** (implémentation naïve) : `T²` par tête. Exemple : `T = 8 192`, 32 têtes, fp16 = **4 Gio par couche** rien que pour les scores. D'où FlashAttention (ci-dessous).

### Les variantes à connaître

| Variante | Idée | Gain |
|---|---|---|
| **MQA** (*Multi-Query*) | Toutes les têtes de requête partagent **une seule** paire K/V | KV cache divisé par `h` |
| **GQA** (*Grouped-Query*) | Les têtes sont réparties en **groupes**, chaque groupe partage une paire K/V | Compromis qualité/mémoire : Llama-2 70B, Llama 3, Mistral |
| **FlashAttention** | Calcule l'attention **exacte** par blocs (tiling) en gardant les calculs dans la mémoire rapide du GPU, sans écrire la matrice `T × T` | Beaucoup moins de mémoire, plus rapide ; mêmes résultats |
| **Fenêtre glissante** | Chaque token ne regarde que les `W` derniers | Coût linéaire en `T` |
| **Cross-attention** | `Q` vient d'une séquence, `K` / `V` d'une autre (encoder-decoder, multimodal) | Relier deux sources |

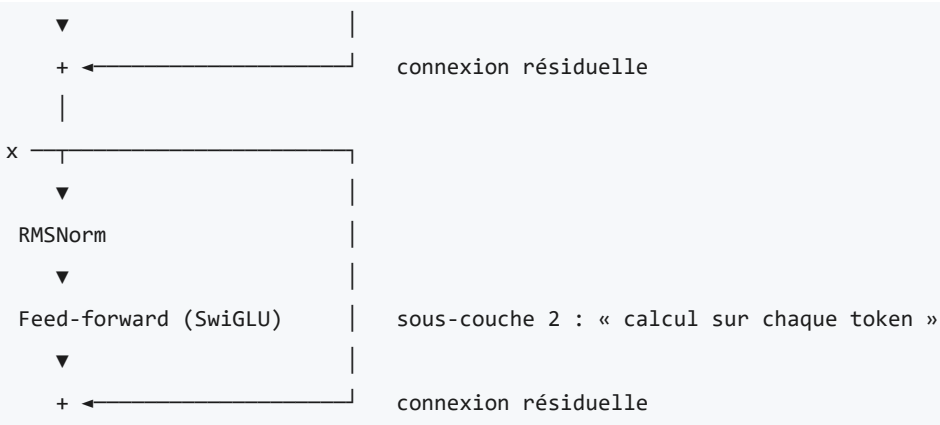
**FlashAttention n'est pas une approximation** : c'est le même calcul, réorganisé pour ménager les lectures/écritures mémoire. Le goulot d'un GPU est souvent la **bande passante mémoire**, pas les opérations.

---

## 5. Le bloc Transformer et l'architecture complète

### Un bloc (modèle « decoder » moderne, style Llama)






Le modèle empile `N` blocs identiques (32 pour un 7B, 80 pour un 70B).

## Les ingrédients, un par un

**Connexions résiduelles** ( `x + f(x)` ) : chaque sous-couche **ajoute** une correction au flux principal. Elles permettent aux gradients de traverser des dizaines de couches et rendent l'entraînement profond possible. On peut voir le modèle comme un « flux » (*residual stream*) que chaque couche lit et enrichit.

**Normalisation** : stabilise les activations.

- *LayerNorm* : soustrait la moyenne, divise par l'écart type, puis échelle et décale.
- **RMSNorm** (Llama) : divise seulement par la racine de la moyenne des carrés. Plus simple, aussi efficace.
- **Pre-norm** (norme *avant* la sous-couche, comme ci-dessus) est plus stable pour les modèles profonds que *post-norm* (norme après, Transformer original).

**Feed-forward (FFN)** : appliqué **indépendamment à chaque token**. Il élargit la dimension, applique une non-linéarité, puis la réduit.

- Classique : `W2 · GELU(W1 · x)` avec `d_ff = 4d` .
- **SwiGLU** (Llama) : `W_down · ( SiLU(W_gate · x) ⊙ (W_up · x) )` , trois matrices, avec `d_ff ≈ 8/3 · d` pour garder le même nombre de paramètres.
- Le FFN contient **les deux tiers des paramètres** d'un bloc et stocke une grande partie des « connaissances » factuelles du modèle.

**Attention** = les tokens échangent de l'information. **FFN** = chaque token traite l'information reçue. Les deux alternent.

## Les trois familles d'architectures

| Famille | Attention | Objectif d'entraînement | Exemples | Usage |
|---|---|---|---|---|
| **Encoder-only** | Bidirectionnelle (voit tout) | Prédire des mots masqués (MLM) | BERT, RoBERTa | Classification, embeddings, reranking |
| **Decoder-only** | Causale (voit le passé) | Prédire le token suivant | GPT, Llama, Mistral, Qwen, Claude | **Génération** : presque tous les LLM |

| Famille | Attention | Objectif d'entraînement | Exemples | Usage |
|---|---|---|---|---|
| **Encoder-decoder** | Encoder bidirectionnel + decoder causal avec cross-attention | Reconstruction, traduction | T5, BART | Traduction, résumé |

**Pourquoi le decoder-only domine** : un seul objectif simple (le token suivant), s'entraîne sur tout texte brut, passe très bien à l'échelle, et gère nativement le prompting et le few-shot.

**Lien avec ton projet RAG** : le modèle d'embedding E5 et le reranker (cross-encoder) sont des **encoders** ; le LLM de génération est un **decoder**.

### Le modèle complet (vérifié)

```
class TinyGPT(nn.Module):
    def __init__(self, vocab_size, d_model=64, n_heads=4, n_layers=2, d_ff=172, max_len=256):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_len, d_model)
        self.blocks = nn.ModuleList(Block(d_model, n_heads, d_ff) for _ in range(n_layers))
        self.norm = RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        self.lm_head.weight = self.tok_emb.weight          # weight tying
```

Le fichier `labs/mini_transformer_lora.py` contient le modèle complet (≈ 120 000 paramètres) avec `RMSNorm`, `SwiGLU` et `Block`. **Lis-le ligne par ligne** : c'est la meilleure préparation d'entretien.

---

## 6. Compter les paramètres et le calcul

### Paramètres d'un bloc (attention multi-têtes classique, FFN à `4d`)

```
attention : W_Q, W_K, W_V, W_O  = 4 × d²
FFN       : d → 4d → d           = 8 × d²
bloc                             ≈ 12 × d²
modèle    ≈ 12 × N × d²  +  V × d   (embeddings)
```

**Vérification sur GPT-2 small** ( `d = 768` , `N = 12` , `V = 50 257` ) : `12 × 12 × 768² ≈ 84,9 M` ; embeddings `50 257 × 768 ≈ 38,6 M` ; positions `1 024 × 768 ≈ 0,8 M` ; total **≈ 124 M**, ce qui correspond bien aux 124 M annoncés.

### Un vrai modèle : Llama-2 7B (SwiGLU, sans GQA)

`d = 4 096` , `N = 32` , `d_ff = 11 008` , `V = 32 000` .

```
attention par couche : 4 × d²          = 4 × 4096²            =  67,1 M
FFN par couche       : 3 × d × d_ff    = 3 × 4096 × 11008     = 135,3 M
une couche                                                     = 202,4 M
32 couches                                                     = 6 476 M
embeddings + sortie (non partagés) : 2 × 32000 × 4096          =  262 M
total                                                          ≈ 6,74 milliards
```

Le résultat (6,74 G) correspond bien au « 7B ». **Dans un bloc, le FFN pèse environ deux fois plus que l'attention.** Garde ce détail : il explique où LoRA applique ses adaptateurs (module 12).

### Le calcul (FLOPs)

- **Passe avant** : environ `2 × N_params` FLOPs **par token**.
- **Entraînement** (avant + arrière) : environ `6 × N_params` FLOPs par token.
- Donc `calcul total ≈ 6 × N × D` pour `D` tokens d'entraînement (règle à connaître).

### Les deux régimes du GPU

- **Limité par le calcul** (*compute-bound*) : beaucoup d'opérations par octet lu (entraînement, prompt de grande taille).
- **Limité par la mémoire** (*memory-bound*) : on lit beaucoup d'octets pour peu d'opérations (génération token par token : on relit tous les poids pour un seul token). **C'est le cas de l'inférence**, ce qui explique pourquoi la quantification accélère la génération (voir module 8).

---

## 7. Le pré-entraînement

### L'objectif

Minimiser la **perte d'entropie croisée** de la prédiction du token suivant :

```
L = − (1/T) Σ log P(token_t | tokens_<t)
```

- **Perplexité** = `exp(L)` : « sur combien de tokens équiprobables le modèle hésite-t-il en moyenne ».
- **Repère de vérification** : un modèle non entraîné a une probabilité uniforme `1/V`, donc une perte initiale de `ln(V)`. Dans notre mini-modèle ( `V = 30` ), la perte initiale mesurée est **3,41**, et `ln 30 = 3,40`. **Si ta perte initiale est très différente de `ln V`, ton initialisation ou ton code est faux.** (Avant d'avoir corrigé l'initialisation des embeddings, la perte initiale du lab était de 47,9.)

### Les données

Des milliers de milliards de tokens : pages web filtrées, livres, code, articles, conversations. Étapes : extraction du texte, **filtrage de qualité**, **déduplication** (essentielle : les doublons font mémoriser), mélange des sources (« mixture »), tokenisation. **La qualité et la composition des données comptent autant que la taille du modèle.**

### L'optimisation (repères de la pratique)

| Élément | Choix courant | Pourquoi |
|---|---|---|
| Optimiseur | **AdamW** ( $\beta 1 = 0{,}9$ , $\beta 2 \approx 0{,}95$ , weight decay ≈ 0,1) | Adaptatif par paramètre, robuste |
| Taux d'apprentissage | Échauffement (*warmup*) linéaire puis décroissance en cosinus | Éviter de diverger au début |
| Écrêtage des gradients | Norme maximale ≈ 1,0 | Éviter les pics (*loss spikes*) |
| Précision | **bf16** (calcul) + copie fp32 (poids maîtres) | Rapide et stable (module 9) |
| Taille de batch | Millions de tokens | Gradients peu bruités, bon usage du matériel |

### Les lois d'échelle

- **Kaplan et al. (2020)** : la perte diminue de façon **régulière et prévisible** (loi de puissance) quand on augmente le nombre de paramètres, les données et le calcul.
- **Chinchilla (Hoffmann et al., 2022)** : pour un budget de calcul donné, taille du modèle et nombre de tokens doivent croître ensemble ; repère : **environ 20 tokens d'entraînement par paramètre**. Beaucoup de modèles de l'époque étaient « sous-entraînés ».
- **En pratique** (modèles ouverts récents) : on **sur-entraîne** volontairement des modèles plus petits sur bien plus de 20 tokens/paramètre. Raison : le coût d'entraînement est payé **une fois**, mais le coût d'**inférence** est payé à chaque requête ; un petit modèle très entraîné est moins cher à servir.
- **Exemple** (calcul) : un modèle de 7 G de paramètres entraîné « Chinchilla-optimal » = 140 G tokens ; calcul $6 \times N \times D \approx 5{,}9 \times 10^{21}$ FLOPs. En le sur-entraînant sur 2 000 G de tokens : $8{,}4 \times 10^{22}$ FLOPs (14 fois plus), pour un modèle meilleur à taille égale.

### Ce que tu ne feras (presque) jamais, mais dois comprendre

Pré-entraîner un modèle de base demande des milliers de GPU. Ton travail : choisir un modèle de base ouvert adapté, puis l'**adapter** (partie II). Mais le **pré-entraînement continu** (poursuivre sur un corpus de domaine) existe pour de grandes adaptations de vocabulaire ou de langue.

---

## 8. La génération et l'inférence

### De la distribution au token

À chaque pas, le modèle produit des **logits** (un score par token du vocabulaire). Comment choisir ?

| Stratégie | Principe | Effet |
|---|---|---|
| **Glouton** (*greedy*) | Toujours le token le plus probable | Déterministe, répétitif |

| Stratégie | Principe | Effet |
|---|---|---|
| **Température** `T` | On divise les logits par `T` avant le softmax | `T < 1` : plus tranché ; `T > 1` : plus aléatoire ; `T → 0` : glouton |
| **Top-k** | On ne garde que les `k` tokens les plus probables | Coupe la « queue » improbable |
| **Top-p** (noyau) | On garde le plus petit ensemble de tokens dont la probabilité cumulée dépasse `p` | S'adapte : peu de choix si le modèle est sûr, plus s'il hésite |
| **Beam search** | On suit plusieurs séquences candidates en parallèle | Traduction, résumé ; rarement en chat |
| **Pénalité de répétition** | On pénalise les tokens déjà produits | Évite les boucles |

Ordre usuel : température, puis top-k, puis top-p, puis tirage. Pour du **factuel ou de l'extraction**, température basse (0 à 0,2). Pour de la créativité, 0,7 à 1.

```
def filter_logits(logits, temperature=1.0, top_k=0, top_p=1.0):      # vérifié
    logits = logits / max(temperature, 1e-8)
    if top_k > 0:
        kth = torch.topk(logits, min(top_k, logits.numel())).values[-1]
        logits = logits.masked_fill(logits < kth, float("-inf"))
    if top_p < 1.0:
        sorted_logits, order = torch.sort(logits, descending=True)
        probs = sorted_logits.softmax(-1)
        remove = probs.cumsum(-1) - probs > top_p     # garde le plus petit noyau dépassant top_p
        sorted_logits = sorted_logits.masked_fill(remove, float("-inf"))
        logits = torch.full_like(logits, float("-inf")).scatter(0, order, sorted_logits)
    return logits
```

Vérifié : `top_k=1` ne garde qu'un token ; `top_p=0,5` sur une distribution dominée par un token n'en garde qu'un ; une température basse **durcit** la distribution et une température haute l'**aplatit**.

## Deux phases : prefill et decode

1. **Prefill** : le modèle lit **tout le prompt en parallèle** (un gros calcul, limité par le calcul). Il produit le premier token. C'est ce qui détermine le **TTFT** (*time to first token*).
2. **Decode** : un token à la fois, **séquentiellement** (limité par la mémoire). C'est ce qui détermine le débit en **tokens par seconde**.

## Le KV cache : indispensable à connaître

Sans cache, pour produire le token `n+1`, on recalculerait les `K` et `V` des `n` tokens précédents. **Mais ils ne changent pas** (masque causal). On les **stocke** : à chaque pas on ne calcule que les `q`, `k`, `v` du **nouveau** token, et on réutilise le cache pour les anciens.

- Sans cache : le coût de génération de `n` tokens est `O(n²)` en recalcul. Avec cache : chaque pas ne traite qu'un token.
- **Propriété vérifiée** : générer avec ou sans cache donne **exactement les mêmes tokens**.

**Taille du KV cache** (formule à retenir) :

```
octets par token = 2 (K et V) × N_couches × N_têtes_KV × dim_tête × octets_par_valeur
```

| Modèle (exemple) | Calcul | Par token | Pour 4 096 tokens |
|---|---|---|---|
| Llama-2 7B (32 couches, 32 têtes KV, dim 128, fp16) | 2 × 32 × 32 × 128 × 2 | **512 Kio** | **2 Gio** |
| Même modèle avec GQA à 8 têtes KV | 2 × 32 × 8 × 128 × 2 | 128 Kio | 0,5 Gio |
| Type 70B avec GQA (80 couches, 8 têtes KV, dim 128) | 2 × 80 × 8 × 128 × 2 | 320 Kio | 10 Gio pour 32 k tokens |

**Le KV cache est par requête** : avec 50 utilisateurs simultanés à 4 096 tokens, un 7B sans GQA consomme **100 Gio** de cache. C'est souvent le vrai facteur limitant du serving, d'où GQA, la quantification du cache et PagedAttention.

### Les optimisations de serving

| Technique | Principe |
|---|---|
| **Batching continu** | On ajoute et retire des requêtes du lot **à chaque pas** au lieu d'attendre la plus lente |
| **PagedAttention** (vLLM) | Le KV cache est découpé en pages, comme la mémoire virtuelle d'un OS : moins de gaspillage et de fragmentation |
| **FlashAttention** | Attention exacte, économe en mémoire |
| **Quantification** | Poids en int8/int4 : moins de mémoire à lire = decode plus rapide |
| **Décodage spéculatif** | Un petit modèle « brouillon » propose plusieurs tokens, le gros modèle les **vérifie en une passe** ; résultat identique, plus rapide |
| **Cache de préfixe** | Les prompts qui partagent un début réutilisent le même KV cache (voir le module 16 de la formation RAG) |
| **Sorties contraintes** | Une grammaire impose un JSON valide en filtrant les tokens autorisés |

### Les métriques de latence

- **TTFT** : délai avant le premier token (dépend du prefill).
- **TPOT** / débit : temps par token de sortie (dépend du decode).
- **Débit global** (tokens/s sur tout le serveur) : ce qui détermine le coût par token.

---

## 9. La mémoire GPU et l'entraînement à grande échelle

### La mémoire en inférence

```
mémoire ≈ poids + KV cache + activations (petit)
poids   = nombre de paramètres × octets par paramètre
```

| Précision | Octets / param. | Modèle 7B (6,74 G) | Remarque |
|---|---|---|---|
| fp32 | 4 | 27 Go | Entraînement historique, rarement en inférence |
| fp16 / **bf16** | 2 | **13,5 Go** | Standard |
| int8 | 1 | 6,7 Go | Peu de perte |
| int4 | 0,5 | **3,4 Go** | Perte modérée ; base de QLoRA |

(On parle ici de 1 Go = $10^9$ octets ; les poids ne sont jamais seuls : ajoute le KV cache et une marge.)

## Pourquoi bf16 plutôt que fp16

Les deux occupent 16 bits. **bf16** garde l'étendue d'exposants de fp32 (moins de précision de mantisse mais **pas de dépassement**), donc stable sans « loss scaling ». **fp16** a une plage étroite et demande un facteur d'échelle de la perte. Sur du matériel récent, on utilise bf16.

## La mémoire en entraînement complet (le chiffre qui motive LoRA)

Pour chaque paramètre, avec Adam et précision mixte :

```
poids bf16 (2) + gradient bf16 (2) + copie maître fp32 (4) + moment m fp32 (4) + moment v fp32 (4)
= 16 octets par paramètre   (+ les activations, qui dépendent du batch et de la longueur)
```

| Modèle | Poids seuls (bf16) | Entraînement complet (16 o/param.) |
|---|---|---|
| 7B (6,74 G) | 13,5 Go | **≈ 108 Go** |
| 70B | 140 Go | **≈ 1,1 To** |

Un 7B ne tient donc pas en entraînement complet sur un GPU de 24 Go, ni même de 80 Go sans partage. **C'est le problème que LoRA résout** : on n'entraîne que quelques dizaines de millions de paramètres, donc gradients et états d'Adam ne concernent que ceux-là (module 12).

## Réduire la mémoire d'entraînement

| Technique | Principe | Coût |
|---|---|---|
| **Accumulation de gradients** | Plusieurs petits micro-batchs avant la mise à jour : gros batch effectif, peu de mémoire | Plus lent |
| **Gradient checkpointing** | On ne garde pas toutes les activations : on les **recalcule** pendant la passe arrière | De l'ordre de 30 % de calcul en plus |
| **Précision mixte** | Calculs en bf16 | Quasi aucun |
| **Optimiseurs 8 bits** | États d'Adam quantifiés | Faible |
| **Découpage du modèle** (ci-dessous) | Répartir sur plusieurs GPU | Communication |

## Entraîner sur plusieurs GPU

| Parallélisme | Ce qu'on découpe | Remarque |
|---|---|---|
| **Données** (DDP) | Le batch ; chaque GPU a une **copie complète** du modèle | Simple ; limité par la taille du modèle |
| **ZeRO / FSDP** | On **partage** entre GPU les états d'optimiseur (ZeRO-1), + gradients (ZeRO-2), + poids (ZeRO-3) | Le modèle n'est plus répliqué ; standard pour fine-tuner de gros modèles |
| **Tenseur** | Les matrices de chaque couche | Très rapide entre GPU reliés par NVLink |
| **Pipeline** | Les couches (GPU 1 : couches 1-20, etc.) | « Bulles » d'inactivité à gérer |
| **Experts** (MoE) | Les experts sur différents GPU | Voir module 19 |

Les très gros pré-entraînements combinent plusieurs de ces formes (« parallélisme 3D »). **Pour un ingénieur** : retiens DDP, FSDP/ZeRO et l'idée qu'on échange mémoire contre communication.

---

# PARTIE II : ADAPTER UN LLM

## 10. Prompting, RAG ou fine-tuning : choisir

### Les quatre leviers, du moins au plus coûteux

```
1. Prompting      (instructions, exemples)          -> minutes, gratuit
2. RAG            (documents fournis à la requête)  -> jours
3. Fine-tuning    (LoRA, etc.)                      -> jours à semaines, données + GPU
4. Pré-entraînement continu / from scratch          -> semaines à mois, très cher
```

**Règle d'or : monter dans l'échelle seulement si l'échelon inférieur ne suffit pas, et le prouver avec une évaluation.** La plupart des projets s'arrêtent à 1 ou 2.

### Ce que fait chaque levier

| Besoin | Meilleur levier | Pourquoi |
|---|---|---|
| Connaissances **privées, changeantes, citables** | **RAG** | Mise à jour sans ré-entraîner, sources vérifiables, droits d'accès |
| **Format de sortie** strict (JSON, structure) | Prompt + sorties contraintes, puis fine-tuning | Le fine-tuning rend le format fiable sans longs exemples |
| **Style / ton / persona** | Fine-tuning (ou bon prompt) | C'est un comportement, pas un fait |
| **Vocabulaire ou raisonnement de domaine** (médical, juridique, code d'un langage rare) | Fine-tuning | Change la façon de penser du modèle |
| **Réduire le coût / la latence** | Fine-tuner un **petit** modèle pour égaler un gros sur une tâche étroite | Prompt plus court, modèle moins cher |
| **Respecter des consignes complexes** de façon fiable | Fine-tuning (SFT) | Apprend les règles par l'exemple |
| **Faits nouveaux** et actualité | **RAG**, pas fine-tuning | Le fine-tuning mémorise mal et hallucine |

### Pourquoi le fine-tuning n'est pas fait pour « ajouter des connaissances »

- Le modèle apprend des **motifs** de style et de comportement bien plus facilement que des faits isolés.
- Il peut apprendre à **répondre avec assurance** sans savoir : le risque d'hallucination augmente si on l'entraîne sur des faits qu'il ne maîtrise pas.
- Les faits changent : il faudrait ré-entraîner, et aucune source n'est citée.
- Résumé : **RAG pour ce que le modèle doit SAVOIR, fine-tuning pour ce que le modèle doit FAIRE.**
- Les deux se **combinent** : un modèle fine-tuné pour bien utiliser un contexte RAG (citer, refuser, suivre le format) est souvent la meilleure combinaison.

### Les questions à se poser avant de fine-tuner

1. Ai-je un **prompt de référence** qui échoue ? Sinon, je n'ai rien à améliorer.
2. Ai-je une **évaluation** chiffrée ? Sinon je ne saurai pas si ça marche.
3. Ai-je assez de **données de qualité** ? (centaines à milliers d'exemples, voir module 15)
4. Quel est le **gain attendu** (qualité, coût, latence) et son prix (données, GPU, maintenance) ?
5. Ai-je le **droit** d'utiliser ces données et ce modèle (licences) ?

### Lien avec ton projet RAG

Pistes réalistes : fine-tuner un **petit modèle** pour l'extraction d'exigences en JSON (tâche étroite, validée en code, coût réduit : voir le module 16 de la formation RAG), ou affiner le **reranker** ou le modèle d'**embedding** sur tes paires (question, passage). Ne **pas** fine-tuner pour « apprendre les documents » : c'est le rôle du RAG.

---

## 11. Fine-tuning complet et méthodes PEFT

### Le fine-tuning complet (*full fine-tuning*)

On continue l'entraînement en **mettant à jour tous les poids**.

- Avantage : capacité d'adaptation maximale.
- Inconvénients : mémoire énorme (16 octets par paramètre, module 9), une **copie complète** du modèle par tâche (13,5 Go pour un 7B), risque d'**oubli catastrophique** (le modèle perd des capacités générales), risque de sur-apprentissage sur de petits jeux de données.

### PEFT : *Parameter-Efficient Fine-Tuning*

Idée : **geler** le modèle de base et n'entraîner qu'un **petit** nombre de paramètres (souvent moins de 1 %). Le fichier produit (l'**adaptateur**) pèse quelques dizaines de Mo.

### Les familles de méthodes

| Famille | Méthode | Principe | Inconvénient |
|---|---|---|---|
| **Additive** | **Adapters** (Houlsby, 2019) | Petits modules (réduction → non-linéarité → expansion) **insérés** dans chaque bloc | Ajoutent de la **latence** à l'inférence (couches en plus) |
| **Prompts doux** | Prompt tuning, **prefix tuning**, P-tuning | Entraînent des vecteurs « virtuels » ajoutés à l'entrée (ou aux K/V de chaque couche) | Consomment du contexte ; moins stables ; souvent moins bons |
| **Sélective** | **BitFit** | N'entraîne que les biais | Capacité limitée |
| **Multiplicative** | **$IA^3$** | Apprend des vecteurs qui **multiplient** les activations (K, V, FFN) | Très peu de paramètres ; capacité limitée |
| **Reparamétrisation** | **LoRA** | Apprend une mise à jour de rang faible `ΔW = BA` qu'on peut **fusionner** | Choix du rang et des modules |

**Pourquoi LoRA a gagné** : (1) qualité proche du fine-tuning complet, (2) **aucune latence ajoutée** après fusion, (3) adaptateurs minuscules et échangeables, (4) simple à implémenter, (5) se combine avec la quantification (QLoRA).

---

# 12. LoRA en profondeur

> LoRA : *Low-Rank Adaptation of Large Language Models* (Hu et al., 2021).

## L'hypothèse de départ

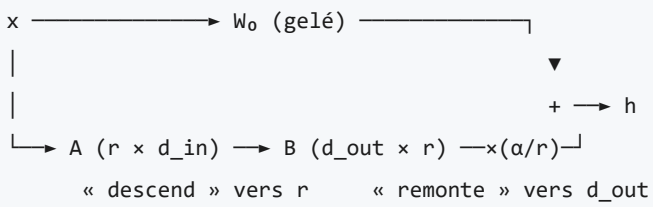
Quand on fine-tune, la **mise à jour des poids** `ΔW = W_final − W_initial` a un **rang intrinsèque faible** : on peut l'approcher par le produit de deux petites matrices. (Appuyé par des travaux sur la « dimension intrinsèque » des tâches de fine-tuning : un modèle peut être adapté en optimisant dans un sous-espace de très petite dimension.)

## La formule

Pour une couche linéaire `h = W₀ x` avec `W₀ ∈ ℝ^(d_out × d_in)` **gelée** :

```
h = W₀ x + ΔW x = W₀ x + (α / r) · B A x
        A ∈ ℝ^(r × d_in)        B ∈ ℝ^(d_out × r)        r ≪ min(d_in, d_out)
```

Seuls `A` et `B` sont entraînés.




## Combien de paramètres : `r × (d_in + d_out)` par matrice adaptée

Au lieu de `d_in × d_out` pour la matrice complète.

| Cas (Llama-2 7B : `d = 4096`, `d_ff = 11008`, 32 couches) | Paramètres entraînés | Part du modèle (6,74 G) |
|---|---|---|
| Une matrice 4096×4096, `r = 8` | 8 × (4096 + 4096) = **65 536** (au lieu de 16,8 M) | 0,4 % de cette matrice |
| `r = 8` sur `q` et `v` (config du papier original) | 2 × 32 × 65 536 = **4,19 M** | **0,06 %** |
| `r = 16` sur **toutes** les couches linéaires | **40,0 M** | **0,59 %** |
| `r = 64` sur toutes les couches linéaires | **159,9 M** | **2,4 %** |

(Toutes les couches linéaires = `q, k, v, o, gate, up, down`.) Taille du fichier d'adaptateur à `r = 16` toutes couches, en bf16 : **≈ 80 Mo**, à comparer aux 13,5 Go du modèle.

## Les trois détails d'initialisation et d'échelle qui font les questions d'entretien

**1. Pourquoi `B = 0` au départ ?**
Pour que `ΔW = BA = 0` : le modèle fine-tuné démarre **exactement** comme le modèle de base (aucune perturbation). `A` est initialisé aléatoirement (Kaiming/gaussien), pas `B`. Si les **deux** étaient nuls, les gradients de `A` et de `B` seraient nuls (chacun dépend de l'autre) et rien n'apprendrait jamais. Si les deux étaient aléatoires, le modèle serait perturbé dès le départ.

**2. À quoi sert `α / r` ?**
C'est un **facteur d'échelle** de la mise à jour. Il permet de **changer `r` sans re-régler le taux d'apprentissage** : avec `α` fixé, augmenter `r` réduit le facteur par adaptateur. Règle courante : `α = r` ou `α = 2r`. `α` n'est pas « de la capacité » : c'est l'amplitude de l'adaptation. (Variante rsLoRA : utiliser `α / √r`, plus stable pour de grands rangs.)

**3. Pourquoi le rang est-il un compromis ?**
Un rang plus grand = plus de capacité (et de paramètres), mais **pas toujours mieux** : au-delà d'un certain seuil, le gain stagne selon la tâche. Pour du style/format, `r = 8` à `16` suffit souvent ; pour des tâches plus difficiles (raisonnement, nouveau domaine), `r = 32` à `128` peut aider.

## Le code (vérifié)

```
class LoRALinear(nn.Module):
    """y = W x + (alpha / r) * B (A x), avec W gelé, A (r x in) et B (out x r) entraînés."""

    def __init__(self, base: nn.Linear, r=8, alpha=16.0, dropout=0.0):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad_(False)                       # le modèle de base est gelé
        self.r, self.scale = r, alpha / r
        self.A = nn.Parameter(torch.empty(r, base.in_features))
        self.B = nn.Parameter(torch.zeros(base.out_features, r))     # B = 0 au départ
        nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))             # comme PEFT
        self.dropout = nn.Dropout(dropout)
        self.merged = False

    def forward(self, x):
        out = self.base(x)
        if self.merged:
            return out
        return out + self.scale * (self.dropout(x) @ self.A.T @ self.B.T)

    @torch.no_grad()
    def merge(self):                                          # W <- W + (alpha/r) B A
        if not self.merged:
            self.base.weight += self.scale * (self.B @ self.A)
            self.merged = True
```

```
    @torch.no_grad()
    def unmerge(self):
        if self.merged:
            self.base.weight -= self.scale * (self.B @ self.A)
            self.merged = False


def apply_lora(model, targets=("qkv", "proj"), r=8, alpha=16.0, dropout=0.0):
    for p in model.parameters():
        p.requires_grad_(False)                          # on gèle TOUT
    for module in list(model.modules()):
        for name, child in list(module.named_children()):
            if isinstance(child, nn.Linear) and name in targets:
                setattr(module, name, LoRALinear(child, r, alpha, dropout))
    return model
```

## Ce que le lab démontre (tout est vérifié par des assertions)

1. **Au départ, sortie identique** au modèle de base ( `B = 0` ).
2. **Nombre de paramètres entraînés = formule** `r × (d_in + d_out)` par matrice.
3. **Expérience** : on pré-entraîne un mini-modèle sur la suite `n, n+1, n+2...`, puis on le fine-tune avec LoRA sur `n, n+2, n+4...` : la perte passe de **5,97 à 0,02** en n'entraînant que **3 072 paramètres sur 120 512 (2,5 %)**.
4. **Seuls** `A` **et** `B` **ont changé** : tous les autres poids sont identiques bit à bit.
5. **La fusion est exacte** : sortie avant fusion = sortie après fusion ; `unmerge` restaure le modèle.

## Pourquoi LoRA économise de la mémoire (et pas tout le calcul)

- **Économisé** : gradients et états d'Adam des poids gelés. Pour un 7B à `r = 16` toutes couches : adaptateurs bf16 ≈ 80 Mo, gradients ≈ 80 Mo, états Adam fp32 ≈ 320 Mo : **moins de 0,5 Go** au lieu de ~108 Go de besoin total en entraînement complet (le gros du coût restant : les poids du modèle).
- **Non économisé** : les **activations**. La passe arrière doit quand même traverser les poids gelés pour propager les gradients aux couches précédentes. Donc LoRA n'est pas dramatiquement plus rapide ; il est surtout **beaucoup moins gourmand en mémoire**. Pour les activations, on utilise le *gradient checkpointing*.

## Où appliquer LoRA ?

- Papier original : seulement `W_q` et `W_v` (expériences sur l'attention).
- **Pratique actuelle** : **toutes les couches linéaires** ( `q, k, v, o` + `gate, up, down` ). Le papier QLoRA a montré que cibler **toutes** les couches linéaires est important pour égaler la qualité du fine-tuning complet, et que cela compte plus que le choix précis du rang.
- Dans PEFT : `target_modules="all-linear"` (hors couche de sortie).
- Le FFN contient la majorité des paramètres : l'omettre limite la capacité.

## Les hyperparamètres (repères, à valider par la mesure)

| Paramètre | Valeurs courantes | Remarque |
|---|---|---|
| Rang `r` | 8, 16, 32, 64 | Commence à 16 ; monte si le modèle sous-apprend |
| `lora_alpha` | `r` ou `2r` | Si tu doubles `r`, vérifie l'échelle `α/r` |
| `lora_dropout` | 0 à 0,1 (souvent 0,05) | Régularisation, utile sur petits jeux de données |
| Modules cibles | tous les linéaires | Moins = moins de capacité |
| Taux d'apprentissage | **≈ 1e-4 à 2e-4** (bien plus haut qu'un fine-tuning complet, ≈ 1e-5) | Trop haut : instable ; trop bas : n'apprend pas |
| Époques | 1 à 3 | Plus = risque de mémorisation |
| Taille de batch effective | 16 à 64 (avec accumulation) | |
| Planification | Warmup ≈ 3 % puis cosinus | |
| Longueur max | 1 024 à 4 096 | Détermine la mémoire des activations |

**Pourquoi un taux d'apprentissage plus haut qu'en fine-tuning complet ?** On part de `B = 0` et on entraîne peu de paramètres : il faut des pas plus grands pour déplacer le modèle de façon utile.

## Inférence : fusionner ou non

| Mode | Fonctionnement | Quand |
|---|---|---|
| **Fusionné** | `W ← W + (α/r)BA`, on enregistre un modèle normal | Un seul usage ; **aucune latence ajoutée** |
| **Non fusionné** | Le chemin LoRA est calculé en plus (légère surcharge) | **Plusieurs adaptateurs** sur un même modèle de base, changés à la volée |

Service **multi-adaptateurs** : un seul modèle de base en mémoire, des dizaines d'adaptateurs (un par client ou par tâche), choisis **par requête** (supporté par des serveurs comme vLLM). Immense économie par rapport à un modèle complet par client.

## Les limites de LoRA (honnêteté attendue en entretien)

- **« LoRA apprend moins et oublie moins »** (étude de comparaison avec le fine-tuning complet) : sur des tâches difficiles qui demandent d'**acquérir beaucoup de nouvelles capacités** (code, maths) à partir de gros volumes de données, LoRA peut rester **derrière** le fine-tuning complet ; en revanche il préserve mieux les capacités d'origine.
- **Sensible aux hyperparamètres** (taux d'apprentissage, modules cibles).
- **Le rang n'est pas toujours le goulot** : parfois la qualité des données ou le taux d'apprentissage comptent davantage.
- Ne résout pas le problème de **connaissances factuelles** (voir module 10).

### Dérivation rapide des gradients (bonus d'entretien)

Avec `h = W₀x + s·BAx` ( `s = α/r` ) et `g = ∂L/∂h` :

```
∂L/∂B = s · g (A x)ᵀ          (taille d_out × r)
∂L/∂A = s · Bᵀ g xᵀ           (taille r × d_in)
∂L/∂x = W₀ᵀ g + s · Aᵀ Bᵀ g
```

À `B = 0` : `∂L/∂A = 0` mais `∂L/∂B ≠ 0`. **B apprend d'abord**, puis `A` reçoit des gradients non nuls. C'est pourquoi l'initialisation « une des deux à zéro » suffit.

---

## 13. QLoRA et les variantes de LoRA

### QLoRA : fine-tuner un gros modèle sur un seul GPU

*QLoRA* (Dettmers et al., 2023) combine **quantification 4 bits du modèle gelé** et **adaptateurs LoRA en 16 bits**. Les gradients traversent les poids 4 bits gelés vers les adaptateurs.

Trois innovations :

| Innovation | Explication |
|---|---|
| **NF4** (*4-bit NormalFloat*) | Un format 4 bits dont les 16 valeurs sont placées pour correspondre à la **distribution normale** des poids : meilleure précision qu'un int4 uniforme à même taille |
| **Double quantification** | Les **constantes de quantification** (une par bloc de poids) sont elles-mêmes quantifiées : économie d'environ 0,4 bit par paramètre |
| **Optimiseurs paginés** | Les états d'Adam peuvent « déborder » vers la RAM CPU lors de pics de mémoire, évitant les erreurs *out of memory* |

**Fonctionnement** : les poids sont stockés en NF4 ; pour chaque calcul, un bloc est **déquantifié à la volée en bf16**, utilisé, puis jeté. Les adaptateurs LoRA et leurs gradients restent en 16 bits.

**Résultat annoncé dans le papier** : fine-tuner un modèle de **65 milliards de paramètres sur un seul GPU de 48 Go**, avec une qualité équivalente au fine-tuning 16 bits complet sur les benchmarks étudiés.

### Mémoire pour un 7B avec QLoRA (ordre de grandeur, à mesurer)

| Poste | Taille |
|---|---|
| Poids NF4 (6,74 G × ~0,5 octet) | ≈ 3,4 Go |
| Adaptateurs LoRA, gradients, états Adam (r = 16, toutes couches) | < 0,5 Go |
| Activations | Dépend du batch et de la longueur (réduit par le gradient checkpointing) |

Cela permet d'entraîner un 7B sur un GPU grand public de 12 à 16 Go avec des séquences modérées. **Contrepartie** : la déquantification à la volée **ralentit** l'entraînement par rapport à LoRA en 16 bits.

### Variantes de LoRA

| Variante | Idée | Quand y penser |
|---|---|---|
| **DoRA** | Décompose le poids en **magnitude** et **direction** ; LoRA n'adapte que la direction, la magnitude est apprise à part | Gain de qualité fréquent à faible rang ; un peu plus coûteux |
| **rsLoRA** | Échelle `α/√r` au lieu de `α/r` | Rangs élevés (64 et plus) |
| **LoRA+** | **Deux taux d'apprentissage** : plus grand pour `B` que pour `A` | Apprentissage plus rapide et meilleur |
| **AdaLoRA** | **Alloue** le rang dynamiquement entre matrices selon leur importance | Budget de paramètres limité |
| **VeRA** | `A` et `B` aléatoires **partagés et gelés** ; on n'apprend que de petits vecteurs d'échelle | Moins de paramètres que LoRA |
| **LoftQ / initialisation quantifiée** | Initialise LoRA pour **compenser l'erreur de quantification** | Meilleur départ pour QLoRA |
| **PiSSA** | Initialise `A`, `B` avec les **composantes principales** (SVD) de `W` | Convergence plus rapide |
| **LoRA multiples / fusion** | Combiner plusieurs adaptateurs (somme pondérée, TIES, DARE) | Mélanger des compétences |

**Comment répondre à « quelle variante choisir ? »** : « Je commence par LoRA ou QLoRA standard avec tous les modules linéaires. Je ne passe à une variante que si l'évaluation montre une limite, et je compare sur la même évaluation. »

---

## 14. Fine-tuner en pratique avec Hugging Face

### La pile d'outils

| Bibliothèque | Rôle |
|---|---|
| `transformers` | Modèles, tokenizers, `Trainer` |
| `datasets` | Chargement et traitement des données |
| `peft` | LoRA et autres méthodes PEFT |
| `trl` | Entraîneurs `SFTTrainer`, `DPOTrainer`, `GRPOTrainer`… |
| `bitsandbytes` | Quantification 4/8 bits (QLoRA) |
| `accelerate` | Multi-GPU, précision mixte |
| Autres | Axolotl, Unsloth, LLaMA-Factory, torchtune : surcouches qui simplifient et accélèrent |

## SFT avec QLoRA (non exécuté ici : nécessite un GPU et ces bibliothèques)

```
import torch
from datasets import load_dataset
from peft import LoraConfig
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from trl import SFTConfig, SFTTrainer

model_id = "..."   # un modèle ouvert de ton choix (vérifie sa licence)

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
)
model = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb, device_map="auto")
tokenizer = AutoTokenizer.from_pretrained(model_id)

lora = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules="all-linear", task_type="CAUSAL_LM",
)

dataset = load_dataset("json", data_files="train.jsonl", split="train")   # champ "messages"

args = SFTConfig(
    output_dir="out",
    num_train_epochs=2,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,          # batch effectif = 16
    learning_rate=2e-4,
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    bf16=True,
    gradient_checkpointing=True,
    logging_steps=10,
    save_strategy="epoch",
)
trainer = SFTTrainer(model=model, args=args, train_dataset=dataset,
                     peft_config=lora, processing_class=tokenizer)
trainer.train()
trainer.model.save_pretrained("out/adapter")        # ne sauvegarde que l'adaptateur (quelques
dizaines de Mo)
```

Remarques de version : le nom de l'argument de longueur maximale ( `max_seq_length` ou `max_length` ) et celui du tokenizer ( `tokenizer` ou `processing_class` ) **varient selon la version de** `trl` . Lis la doc de ta version.

## Fusionner l'adaptateur et exporter (non exécuté ici)

```
from peft import PeftModel

base = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.bfloat16)   # en 16 bits, PAS
en 4 bits
model = PeftModel.from_pretrained(base, "out/adapter")
merged = model.merge_and_unload()
merged.save_pretrained("merged"); tokenizer.save_pretrained("merged")
```

**Pourquoi recharger en 16 bits pour fusionner** : fusionner une mise à jour fine dans des poids déjà arrondis en 4 bits perd de la précision. On fusionne en bf16, puis on **re-quantifie** si besoin (module 18).

## Le format des données et le *chat template*

Les données de conversation sont des listes de messages :

```
{"messages": [
  {"role": "system", "content": "Tu es un assistant de conformité."},
  {"role": "user", "content": "Exigence : chiffrement au repos."},
  {"role": "assistant", "content": "{\"status\": \"COMPLIANT\", ...}"}
]}
```

Le `tokenizer.apply_chat_template(...)` transforme cela en texte avec les **tokens spéciaux propres au modèle**. **Erreur classique** : fine-tuner avec un gabarit et interroger avec un autre ; le modèle se comporte alors mal sans erreur visible.

## Masquer le prompt dans la perte (« completion-only loss »)

On veut apprendre à **produire la réponse**, pas à recopier la question. On met les labels du prompt à `-100` pour que la perte ne porte que sur les tokens de l'assistant. Selon la version de `trl` , cela se règle par une option dédiée ; sinon, un collateur spécifique. **Vérifie toujours** en affichant les labels d'un exemple.

## Packing

Concaténer plusieurs exemples courts dans une même séquence pour remplir le contexte : **accélère** l'entraînement. Attention aux fuites d'attention entre exemples si le masque n'est pas géré.

## Matériel : ordres de grandeur (à mesurer sur ton cas)

| Modèle | Méthode | GPU typique |
|---|---|---|
| 0,5 à 3 G | LoRA ou QLoRA | 8 à 16 Go (peut tourner sur un GPU gratuit de notebook) |
| 7 à 8 G | QLoRA | 12 à 24 Go |

| Modèle | Méthode | GPU typique |
|---|---|---|
| 7 à 8 G | LoRA 16 bits | 24 à 48 Go |
| 13 à 34 G | QLoRA | 24 à 48 Go |
| 70 G | QLoRA | 48 à 80 Go |
| 7 G | Fine-tuning **complet** | Plusieurs GPU de 80 Go (FSDP/ZeRO) |

### Diagnostiquer un entraînement

| Symptôme | Causes probables | Remède |
|---|---|---|
| Perte qui **ne baisse pas** | Taux d'apprentissage trop bas/haut, mauvais masquage, données incohérentes | Tester sur 20 exemples (doit sur-apprendre) ; vérifier les labels |
| Perte → **presque 0** | Mémorisation (jeu trop petit, trop d'époques) | Moins d'époques, plus de données, dropout, jeu de validation |
| **NaN** | fp16 instable, taux trop haut | bf16, baisser le taux, écrêtage des gradients |
| Le modèle **ne s'arrête pas** de générer | Token de fin absent des données | Ajouter `eos_token` à chaque exemple |
| Réponses **répétitives ou étranges** | Mauvais gabarit de chat, température | Utiliser le même `chat_template` à l'inférence |
| **Out of memory** | Séquences longues, batch grand | Gradient checkpointing, batch plus petit + accumulation, QLoRA, raccourcir |
| Bon sur le jeu d'entraînement, **mauvais ailleurs** | Sur-apprentissage, jeu non représentatif | Validation séparée, plus de diversité |
| Le modèle a **perdu des capacités générales** | Oubli catastrophique, taux trop haut, trop d'époques | Rang plus faible, moins d'époques, mélanger des données générales |

**Test indispensable** : avant un long entraînement, **sur-apprendre 20 exemples** (la perte doit tendre vers 0). Si ce n'est pas le cas, le pipeline est cassé (données, masque, gabarit, taux).

---

## 15. Les données

### La leçon à retenir : qualité avant quantité

L'étude **LIMA** (2023) a montré qu'un modèle de base solide, affiné sur seulement **1 000 exemples soigneusement choisis**, peut produire des réponses de très bonne qualité : le pré-entraînement apporte les connaissances, le fine-tuning apprend surtout **le format et le style**. (Hypothèse de « l'alignement superficiel ».) Conséquence pratique : 500 exemples excellents valent mieux que 50 000 exemples bruités.

## Combien d'exemples ?

| Objectif | Ordre de grandeur |
|---|---|
| Format / style / persona | 100 à 1 000 |
| Tâche étroite (extraction, classification) | 500 à 5 000 |
| Suivre des instructions variées | 1 000 à 50 000, très diversifiés |
| Nouveau domaine ou nouvelle compétence | Dizaines de milliers et plus ; envisager le pré-entraînement continu |

(Repères, pas des règles : commence petit, évalue, puis ajoute des données là où le modèle échoue.)

## Checklist de qualité

- **Cohérence** : mêmes conventions de format et de ton dans tous les exemples.
- **Exactitude** : les réponses sont vraies et relues (par un expert si possible).
- **Diversité** : couvrir les cas réels, les cas limites, les refus, les questions piégeuses.
- **Représentativité** : même distribution que la production (longueur, langue, difficulté).
- **Déduplication** : exemples identiques ou quasi identiques retirés (sinon mémorisation).
- **Équilibre** : pas de classe dominante qui écrase les autres.
- **Données personnelles** : retirées ou masquées (le fine-tuning peut **mémoriser et restituer**).
- **Pas de contamination** : aucun exemple de test dans l'entraînement.
- **Jeu de validation** séparé (au moins 5 à 10 %), jamais utilisé pour entraîner.

## D'où viennent les données ?

1. **Données réelles** (historique, tickets, documents annotés) : les meilleures, à nettoyer et anonymiser.
2. **Annotation humaine** : coûteuse, fiable ; définir un guide d'annotation.
3. **Données synthétiques** : générées par un modèle plus fort (*distillation*) ou auto-instruction. Avantage : rapide ; risques : **erreurs propagées**, perte de diversité, et **conditions d'utilisation** du modèle qui a servi à générer (certaines interdisent d'entraîner un concurrent). Toujours filtrer et relire un échantillon.
4. **Amélioration par l'échec** : prendre les cas où le modèle actuel se trompe, les corriger, les ajouter. Souvent le meilleur rendement.

## Erreurs courantes

- Entraîner sur des réponses **trop courtes ou trop longues** par rapport à ce qu'on attend en production.
- Oublier les **exemples de refus** (« je ne sais pas ») : le modèle apprend à toujours répondre.
- Mélanger plusieurs styles contradictoires.
- Ne pas versionner le jeu de données (impossible de reproduire un résultat).

---

# 16. L'alignement : SFT, RLHF, DPO, GRPO

## La chaîne de post-entraînement

```
Modèle de base ⟶ SFT (imiter de bonnes réponses) ⟶ Optimisation de préférences (RLHF / DPO) ⟶ Modèle « chat »
```

## SFT : *Supervised Fine-Tuning*

Entraînement classique sur des paires (instruction, réponse idéale), avec la perte sur la réponse. C'est ce qu'on fait avec LoRA dans le module 14. Il apprend **le format et le comportement**.

## RLHF : *Reinforcement Learning from Human Feedback*

Trois étapes :

1. **SFT** initial.
2. **Modèle de récompense** (*reward model*) : des humains classent des paires de réponses (préférée vs rejetée) ; on entraîne un modèle à prédire la préférence avec la perte de Bradley-Terry : `L = −log σ( r(x, y_préférée) − r(x, y_rejetée) )`.
3. **Optimisation par renforcement** (souvent **PPO**) : le modèle génère, le modèle de récompense note, on met à jour la politique pour augmenter la récompense, avec une **pénalité KL** qui empêche de trop s'éloigner du modèle de référence.

Problèmes : **complexe** (4 modèles en mémoire : politique, référence, récompense, valeur), **instable**, **détournement de la récompense** (*reward hacking* : le modèle exploite les failles du juge, par exemple en produisant des réponses longues et flatteuses).

## DPO : *Direct Preference Optimization* (Rafailov et al., 2023)

Idée : on peut **éliminer le modèle de récompense et le renforcement**. On entraîne directement sur des paires (préférée `y_w`, rejetée `y_l`) avec une perte de classification :

```
L_DPO = − log σ( β · [ (log π(y_w|x) − log π_ref(y_w|x))  −  (log π(y_l|x) − log π_ref(y_l|x)) ] )
```

- `π` : le modèle entraîné ; `π_ref` : le modèle de référence (gelé, souvent le SFT).
- `β` : force du maintien près de la référence (≈ 0,1).
- Intuition : on augmente la probabilité de la réponse préférée **relativement à** la référence, et on diminue celle de la rejetée.

```
def dpo_loss(policy_chosen, policy_rejected, ref_chosen, ref_rejected, beta=0.1):    # vérifié
    margin = (policy_chosen - ref_chosen) - (policy_rejected - ref_rejected)
    return -F.logsigmoid(beta * margin).mean()
```

Vérifié : quand la politique est identique à la référence, la perte vaut `ln 2 ≈ 0,693` (aucune préférence apprise) ; elle **diminue** si la politique favorise déjà la bonne réponse et **augmente** si elle favorise la mauvaise.

**Avec LoRA** : un grand avantage pratique : `π_ref` s'obtient **sans copie du modèle** en désactivant simplement les adaptateurs. DPO + LoRA tient sur un seul GPU.

### Autres méthodes à situer

| Méthode | Idée |
|---|---|
| **ORPO** | Fusionne SFT et préférence en une seule étape, sans modèle de référence |
| **KTO** | N'a besoin que de « bon / mauvais » par réponse, pas de paires |
| **SimPO** | Pas de modèle de référence ; utilise la log-probabilité moyenne comme récompense |
| **RLAIF / Constitutional AI** | Les préférences sont données par un **modèle** guidé par des principes écrits, pas par des humains |

### RL avec récompenses vérifiables et GRPO

Pour le **raisonnement** (maths, code), on peut calculer automatiquement si la réponse est **correcte** (*RLVR* : récompenses vérifiables). **GRPO** (*Group Relative Policy Optimization*) :

- pour chaque prompt, on génère un **groupe** de `G` réponses ;
- on calcule leur récompense (juste/faux, format) ;
- l'**avantage** de chaque réponse = `(r − moyenne du groupe) / écart type du groupe` ;
- on renforce celles qui sont meilleures que la moyenne du groupe. **Pas de modèle de valeur (critique)** : plus léger que PPO. C'est la base des modèles de raisonnement ouverts récents.

### Pour l'ingénieur

SFT (LoRA) couvre l'immense majorité des besoins. DPO s'ajoute quand on a des **préférences** (réponse A meilleure que B). Le RL est un sujet avancé.

---

## 17. Évaluer un modèle fine-tuné

### Règle fondamentale : comparer à la bonne référence

Un fine-tuning n'a de valeur que s'il bat **le modèle de base avec un bon prompt** (et éventuellement RAG) sur **ta tâche**. Évalue toujours :

1. le modèle de base avec prompt simple ;
2. le modèle de base avec **le meilleur prompt** (instructions + exemples) ;
3. le modèle fine-tuné. Si (3) ne bat pas nettement (2), le fine-tuning ne vaut pas son coût de maintenance.

## Les familles de mesures

| Mesure | À quoi ça sert | Limites |
|---|---|---|
| **Perte / perplexité** sur un jeu de validation | Détecter sur-apprentissage et divergence | Ne dit pas si les réponses sont **utiles** |
| **Métriques de tâche** (exactitude, F1, JSON valide, exact match) | Tâches à réponse vérifiable | Peu adaptées au texte libre |
| **Benchmarks généraux** (MMLU, GSM8K, HumanEval, etc.) | Détecter l'**oubli** de capacités générales | Risque de contamination ; ne représentent pas ta tâche |
| **LLM-as-judge** | Noter des réponses ouvertes à grande échelle | Biais (préfère les réponses longues, son propre style, la première position) ; à **calibrer** sur des notes humaines |
| **Évaluation humaine** | La référence, par échantillon | Lente et chère |
| **Comparaisons par paires / arènes** | Classer des modèles par préférences (score Elo) | Reflète des préférences générales |
| **Tests de régression** | Cas critiques qui ne doivent jamais casser | À entretenir |

## Ce qu'il faut vérifier après un fine-tuning

- **Qualité sur la tâche** (jeu de test non vu).
- **Oubli catastrophique** : le modèle reste-t-il bon en général ? (quelques benchmarks de contrôle)
- **Sécurité** : le fine-tuning peut **réduire** les garde-fous du modèle ; retester les refus.
- **Format** : validité du JSON, respect des consignes.
- **Hallucinations** sur des questions hors du domaine.
- **Robustesse** : reformulations, entrées longues, entrées bruitées.
- **Coût et latence** (c'est souvent le but).

## Pièges

- Évaluer sur des données **vues pendant l'entraînement** (contamination).
- Conclure à partir de **10 exemples** : trop peu pour distinguer deux modèles.
- Se fier uniquement à la perte de validation.
- Choisir le meilleur checkpoint **sur le jeu de test** (le jeu de test devient un jeu d'entraînement).
- Oublier la **variance** : relancer avec une autre graine ; si l'écart disparaît, ce n'était pas réel.

---

# 18. Quantification, fusion et déploiement

## La quantification pour l'inférence

Représenter les poids avec moins de bits : moins de mémoire, **decode plus rapide** (on lit moins d'octets : l'inférence est limitée par la mémoire).

| Approche | Principe |
|---|---|
| **PTQ** (*post-training quantization*) | On quantifie un modèle déjà entraîné, avec un petit jeu de calibration |
| **QAT** (*quantization-aware training*) | On simule la quantification **pendant** l'entraînement : meilleure qualité, plus coûteux |

| Format / méthode | Remarque |
|---|---|
| **int8** (LLM.int8) | Peu de perte ; traite à part les valeurs aberrantes (*outliers*) |
| **GPTQ** | PTQ 4 bits, utilise des informations du second ordre ; pour GPU |
| **AWQ** | PTQ 4 bits qui protège les poids « importants » selon les **activations** |
| **GGUF** (llama.cpp) | Format pour CPU/Apple/GPU local, niveaux `Q4_K_M`, `Q5_K_M`, `Q8_0` … |
| **NF4** (bitsandbytes) | 4 bits pour **fine-tuning** (QLoRA) |
| **FP8** | Matériel récent ; bon compromis qualité/vitesse |

Repères : 8 bits ≈ sans perte notable ; 4 bits ≈ perte modérée mais acceptable pour beaucoup d'usages ; en dessous, perte nette. **Mesure sur ta tâche.**

## Le flux de déploiement d'un modèle fine-tuné avec LoRA

```
modèle de base + adaptateur LoRA
        │
        ├─→ Option A : fusionner (bf16) → (re-quantifier GPTQ/AWQ/GGUF) → serveur d'inférence
        │
        └─→ Option B : garder l'adaptateur séparé → serveur multi-adaptateurs (un modèle de base, N
adaptateurs)
```

- **Option A** : simple, aucune surcharge, un artefact par version.
- **Option B** : idéale pour plusieurs clients ou tâches ; changement d'adaptateur par requête.
- **Piège** : fusionner dans un modèle déjà quantifié en 4 bits dégrade la précision. Fusionner en 16 bits puis re-quantifier. Vérifier la qualité **après** quantification.

## Les serveurs d'inférence

| Outil | Points forts |
|---|---|
| **vLLM** | Batching continu, PagedAttention, prise en charge de plusieurs adaptateurs LoRA, API compatible OpenAI |
| **TGI** (Hugging Face) | Serveur de production, support LoRA |

| Outil | Points forts |
|---|---|
| **llama.cpp / Ollama** | Local, CPU/Apple Silicon, GGUF |
| **TensorRT-LLM** | Très performant sur GPU NVIDIA, plus complexe |

## Dimensionner un déploiement

```
mémoire GPU ≥ poids (selon la précision) + KV cache × nombre de requêtes simultanées + marge
```

Exemple : 7B en bf16 (13,5 Go) + 32 requêtes de 4 096 tokens (32 × 2 Gio = 64 Go avec 32 têtes KV) : **≈ 80 Go**. Avec GQA (cache ÷4) : ≈ 30 Go. Voilà pourquoi GQA et la quantification du KV cache comptent.

## Coût

Le coût d'un service auto-hébergé = **GPU loué ÷ débit**. Un petit modèle fine-tuné, quantifié, bien batché peut coûter **beaucoup moins par requête** qu'une API de gros modèle, **si le volume est suffisant** pour amortir le GPU. À volume faible, l'API est souvent moins chère. Voir le module 16 de la formation RAG (coût et cache) : les mêmes leviers (cache, routage, tokens) s'appliquent.

## Fusion de modèles (*model merging*)

Combiner plusieurs modèles ou adaptateurs **sans entraînement** : moyenne de poids, **SLERP**, **TIES**, **DARE**. Peut mélanger des compétences, mais le résultat est empirique : à évaluer.

---

# PARTIE III : ALLER PLUS LOIN

## 19. Architectures modernes : MoE, long contexte, multimodal, raisonnement

### Mixture of Experts (MoE)

Idée : remplacer le FFN d'un bloc par **plusieurs FFN « experts »** et un **routeur** qui choisit, pour chaque token, les `k` experts à activer (souvent 2).

- **Paramètres totaux** : élevés (tous les experts). **Paramètres actifs par token** : faibles.
- Exemple : Mixtral 8x7B a environ **47 G de paramètres au total mais ≈ 13 G actifs** par token : la qualité d'un gros modèle pour le **coût de calcul** d'un modèle moyen.
- **Contrepartie** : la **mémoire** reste celle de tous les experts (il faut charger les 47 G) ; le routage demande un équilibrage de charge entre experts.
- **LoRA et MoE** : on adapte en général l'attention (et parfois chaque expert), ce qui multiplie les paramètres.

### Long contexte

- **Extension RoPE** (interpolation de positions, YaRN) + court fine-tuning sur de longues séquences.
- **FlashAttention** pour la mémoire, **GQA** pour le KV cache, **fenêtre glissante** pour borner le coût.
- **Limite** : avoir une grande fenêtre ne garantit pas de **bien utiliser** l'information (« lost in the middle », voir la formation RAG). Mesurer avec des tests de type « aiguille dans une botte de foin ».
- Le coût du contexte est **quadratique** pour l'attention et **linéaire** pour le KV cache.

### Modèles multimodaux

Un **encodeur de vision** (souvent un ViT) transforme l'image en vecteurs ; un **projecteur** les met dans l'espace d'embedding du LLM ; ils sont traités comme des « tokens d'image » devant le texte. On peut les fine-tuner avec LoRA sur le LLM (et parfois le projecteur).

### Modèles de raisonnement

Entraînés (souvent par RL avec récompenses vérifiables, GRPO) à produire une **longue chaîne de raisonnement** avant la réponse (*test-time compute* : dépenser plus de calcul à l'inférence pour de meilleurs résultats).

- Avantage : bien meilleurs sur maths, code, logique.
- Conséquences d'ingénierie : **les tokens de raisonnement sont facturés** et allongent la latence ; avec un plafond de tokens trop bas, la réponse peut être **vide** (cas rencontré dans le projet RAG) ; ils sont **trop** pour des tâches simples.

### Alternatives à l'attention

Les **modèles à espace d'états** (Mamba) et architectures hybrides ont un coût **linéaire** en longueur, sans KV cache croissant. Prometteurs pour le très long contexte ; les Transformers restent dominants.

### Autres sujets à connaître de nom

| Sujet | En une phrase |
|---|---|
| **Distillation** | Entraîner un petit modèle (élève) à imiter un grand (maître) |
| **Décodage spéculatif** | Un petit modèle propose, le grand vérifie : résultats identiques, plus rapide |
| **Appel de fonctions / outils** | Le modèle produit un appel structuré ; le code l'exécute |
| **Agents** | Boucle modèle → outil → observation → modèle ; il faut plafonner étapes et coûts |
| **Embeddings** | Modèles encoder qui représentent un texte par un vecteur (RAG) |

---

## 20. Sécurité, limites et licences

### Limites structurelles d'un LLM

- **Hallucinations** : réponses plausibles mais fausses ; le modèle optimise la vraisemblance, pas la vérité.
- **Pas de mémoire** entre appels : tout ce que le modèle « sait » de la conversation est dans le contexte.
- **Date de coupure** des connaissances.
- **Sensibilité au prompt** : une reformulation peut changer la réponse.
- **Raisonnement fragile** sur l'arithmétique, le comptage de lettres (tokenisation), les longues chaînes.
- **Biais** hérités des données.

### Risques propres au fine-tuning

| Risque | Explication | Parade |
|---|---|---|
| **Érosion de la sécurité** | Fine-tuner, même sur des données inoffensives, peut **affaiblir les refus** appris à l'alignement | Retester les refus ; mélanger des exemples de sécurité |
| **Mémorisation et fuite** | Le modèle peut **restituer** des passages des données d'entraînement (données personnelles, secrets) | Anonymiser, dédupliquer, ne pas entraîner sur des secrets |
| **Oubli catastrophique** | Perte de capacités générales | Rang faible, peu d'époques, données mélangées, évaluation de contrôle |
| **Sur-spécialisation** | Excellent sur la tâche, médiocre ailleurs | Valider le périmètre d'usage |
| **Données empoisonnées** | Des exemples malveillants installent un comportement caché | Contrôler la provenance des données |

### Risques d'application (voir la formation RAG, module 14)

Injection de prompt, fuite de contexte, droits d'accès, journalisation, RGPD. Un modèle fine-tuné **n'est pas** plus sûr : il faut les mêmes protections (validation des sorties en code, relecture humaine, principe du moindre privilège pour les outils).

### Licences : « ouvert » ne veut pas dire « libre »

- **Poids ouverts** (*open weights*) : on peut télécharger et utiliser le modèle, **sous licence** qui peut limiter l'usage commercial, la taille de l'entreprise, ou l'entraînement d'autres modèles.
- **Open source** au sens strict : licence permissive (par exemple Apache 2.0) sur le code **et** les poids.
- Vérifier : usage commercial, redistribution des adaptateurs/modèles dérivés, conditions d'utilisation des sorties (pour la **génération de données synthétiques**).
- Les **données d'entraînement** ont aussi des droits (droit d'auteur, RGPD).

---

## 21. Labs et exercices

### Le lab exécutable

Le fichier `docs/labs/mini_transformer_lora.py` contient un mini-GPT complet, la génération avec KV cache, RoPE, le tokenizer BPE, la perte DPO, LoRA et le calcul de mémoire. Chaque propriété est vérifiée.

```
python docs/labs/mini_transformer_lora.py
```

Sortie attendue (valeurs mesurées sur CPU) :

```
causalité                    OK
KV cache = sans cache        OK
entraînement                 OK (perte 3.41 -> 0.008)
RoPE (position relative)     OK
température / top-k / top-p  OK
LoRA                         OK (3,072 paramètres entraînés sur 120,512, perte 5.97 -> 0.02)
formules de mémoire          OK
BPE                          OK (205 octets -> 15 tokens)
perte DPO (ln 2 au départ)   OK
```

### Exercices progressifs

| # | Exercice | Ce que tu apprends |
|---|---|---|
| 1 | Réécris l'attention **sans regarder** : projections, masque, softmax, multi-têtes | La formule et les formes des tenseurs |
| 2 | Supprime le masque causal : l'entraînement semble-t-il meilleur ? Que se passe-t-il à la génération ? | Pourquoi le masque est indispensable (fuite du futur) |
| 3 | Remplace les positions apprises par **RoPE** dans `TinyGPT` | Positions relatives |

| # | Exercice | Ce que tu apprends |
|---|---|---|
| 4 | Entraîne `TinyGPT` sur un petit texte (caractères) et fais-le générer à des températures 0, 0,7, 1,5 | Effet de l'échantillonnage |
| 5 | Mesure le temps de génération **avec et sans KV cache** pour 200 tokens | Pourquoi le cache est vital |
| 6 | Compare la perte initiale à `ln(V)` pour différentes initialisations | Diagnostic d'un bug d'initialisation |
| 7 | Applique LoRA avec `r = 1, 2, 4, 8` : trace la perte finale | Le compromis rang / capacité |
| 8 | Applique LoRA seulement à `qkv`, puis seulement à `proj`, puis aux deux | Où adapter |
| 9 | Teste `B` initialisé **aléatoirement** (au lieu de zéro) : la sortie initiale diffère-t-elle ? | Pourquoi `B = 0` |
| 10 | Mets **A et B à zéro** : le modèle apprend-il ? | Gradients nuls (dérivation du module 12) |
| 11 | Entraîne **deux adaptateurs** ( `n+2` et `n+3` ) et alterne-les sur le même modèle de base | Service multi-adaptateurs |
| 12 | Implémente DoRA ou LoRA+ (deux taux d'apprentissage) | Variantes |
| 13 | Calcule la mémoire d'entraînement et d'inférence d'un modèle de ton choix (poids, KV cache) | Dimensionnement |
| 14 | **Sur un GPU gratuit de notebook** : fine-tune un petit modèle ouvert (0,5 à 3 G) avec QLoRA sur 200 exemples d'un format précis (ex. JSON de conformité) | Le pipeline réel |
| 15 | Sur-apprends 20 exemples (la perte doit tendre vers 0) | Le test de santé d'un pipeline |
| 16 | Compare base+prompt, base+exemples, fine-tuné sur un jeu de test de 50 exemples | Évaluation honnête |
| 17 | Fusionne l'adaptateur, quantifie en 4 bits, et ré-évalue | Perte due à la quantification |
| 18 | Sers l'adaptateur avec un serveur compatible (ex. vLLM) et mesure TTFT et débit | Serving |

## Projet de synthèse (idéal pour un entretien)

**« Fine-tuner un petit modèle pour l'extraction d'exigences de RFP en JSON »** (lien avec ton projet RAG) :

1. Constituer 300 à 1 000 exemples (RFP → JSON), validés en code.
2. Baseline : un gros modèle avec un bon prompt ; mesurer exactitude, JSON valide, coût, latence.
3. QLoRA sur un petit modèle ; évaluer sur un jeu de test séparé.
4. Comparer qualité **et** coût par requête ; conclure honnêtement (le fine-tuning vaut-il le coup ?).
5. Documenter les limites. **Raconte ce que tu as mesuré, pas ce que tu espérais.**

---

# 22. Plan de révision sur trois semaines

| Jours | Travail |
|---|---|
| 1-2 | Modules 1 à 3 (tokens, embeddings, positions) ; exercice BPE ; lis `labs/mini_transformer_lora.py` jusqu'à l'attention |
| 3-4 | Modules 4 et 5 (attention, bloc) ; exercices 1, 2, 3 |
| 5 | Modules 6 et 7 (paramètres, pré-entraînement) ; refais le calcul de Llama-2 7B de tête |
| 6-7 | Modules 8 et 9 (inférence, KV cache, mémoire) ; exercices 4, 5, 13 |
| 8 | Module 10 et 11 (choisir, PEFT) |
| 9-10 | **Module 12 (LoRA)** : formule, init, gradients, paramètres, hyperparamètres ; exercices 7 à 10 |
| 11 | Module 13 (QLoRA et variantes) ; exercices 11 et 12 |
| 12 | Modules 14 et 15 (pratique, données) ; exercice 15 |
| 13 | Modules 16 à 18 (alignement, évaluation, déploiement) ; exercices 16 à 18 |
| 14 | Module 19 et 20 ; relire l'aide-mémoire (module 23) |
| 15-16 | Banque de questions : fondamentaux, attention, architecture |
| 17-18 | Banque de questions : LoRA, QLoRA, fine-tuning, alignement |
| 19 | Banque de questions : inférence, déploiement, conception de systèmes |
| 20 | Écris **de mémoire** : attention, LoRA, top-p, calcul de KV cache |
| 21 | Simulation d'entretien : « explique le Transformer en 3 minutes », « explique LoRA en 2 minutes », 10 questions au hasard |

# 23. Aide-mémoire : formules et ordres de grandeur

## Formules à connaître par cœur

| Sujet | Formule |
|---|---|
| Probabilité d'une séquence | $P(t_1..t_n) = \Pi\ P(t_i \mid t_1..t_{i-1})$ |
| Attention | $\text{softmax}(Q\ K^T / \sqrt{d_k})\ V$ |
| Perte (entropie croisée) | $L = -(1/T)\ \Sigma\ \log P(t_t \mid t_{<t})$ ; perplexité $= \exp(L)$ |
| Perte initiale (modèle ignorant) | $\ln(V)$ |
| Paramètres d'un Transformer | $\approx 12\ N\ d^2 + V\ d$ (FFN à 4d) |
| Bloc Llama (SwiGLU) | attention $4d^2$ + FFN $3 \cdot d \cdot d_ff$ |

| Sujet | Formule |
|---|---|
| FLOPs par token | passe avant `≈ 2N` ; entraînement `≈ 6N` |
| Calcul total d'entraînement | `≈ 6 N D` |
| Chinchilla | `≈ 20` tokens par paramètre |
| KV cache | `2 × couches × têtes_KV × dim_tête × octets` par token |
| Mémoire des poids | `paramètres × octets par paramètre` (fp16 : 2, int8 : 1, int4 : 0,5) |
| Mémoire d'entraînement complet (Adam, précision mixte) | `≈ 16 octets / paramètre` + activations |
| LoRA | `h = W₀x + (α/r) B A x` , `B = 0` au départ |
| Paramètres LoRA par matrice | `r × (d_in + d_out)` |
| Fusion LoRA | `W ← W + (α/r) B A` |
| Perte DPO | `−log σ(β [(logπ_w − logπ_ref,w) − (logπ_l − logπ_ref,l)])` |
| Avantage GRPO | `(r − moyenne_groupe) / écart_type_groupe` |

## Ordres de grandeur utiles (Llama-2 7B)

| Grandeur | Valeur |
|---|---|
| Paramètres | 6,74 G ( `d = 4096` , 32 couches, `d_ff = 11008` , `V = 32000` ) |
| Poids en bf16 / int4 | 13,5 Go / 3,4 Go |
| Entraînement complet | ≈ 108 Go |
| KV cache | 512 Kio par token ; 2 Gio pour 4 096 tokens (avec GQA à 8 têtes KV : ÷4) |
| LoRA `r = 8` sur `q, v` | 4,19 M paramètres (0,06 %) |
| LoRA `r = 16` toutes couches | 40 M paramètres (0,59 %), ≈ 80 Mo en bf16 |
| Taux d'apprentissage | LoRA ≈ 1e-4 à 2e-4 ; fine-tuning complet ≈ 1e-5 |

## Les réponses courtes à avoir en tête

- **Un LLM ?** Un modèle qui prédit le token suivant ; tout le reste en découle.
- **L'attention ?** Chaque token récupère de l'information des autres par une moyenne pondérée de valeurs, les poids venant de la similarité requête/clé.
- **Pourquoi `√d_k` ?** Garder la variance des scores à 1 pour que le softmax ne sature pas.
- **Pourquoi le masque causal ?** Empêcher de voir le futur, et entraîner toutes les positions en un seul passage.
- **KV cache ?** Stocker K et V des tokens passés pour ne pas les recalculer.
- **LoRA ?** Geler le modèle, apprendre une mise à jour de rang faible `BA` , fusionnable.
- **Pourquoi `B = 0` ?** Départ identique au modèle de base ; `A` aléatoire pour que les gradients circulent.

- **QLoRA ?** LoRA sur un modèle de base quantifié en 4 bits (NF4), pour fine-tuner sur un seul GPU.
- **RAG ou fine-tuning ?** RAG pour savoir, fine-tuning pour faire.

---

## 24. Glossaire

| Terme | Sens |
|---|---|
| **Adaptateur** | Petit module appris et ajouté à un modèle gelé (LoRA produit des adaptateurs) |
| **Alignement** | Entraîner un modèle à suivre des instructions et des préférences humaines |
| **Autorégressif** | Génère un token à la fois en réutilisant les précédents |
| **bf16 / fp16** | Formats de nombres sur 16 bits ; bf16 a la plage de fp32 |
| **BPE** | *Byte Pair Encoding* : algorithme de tokenisation par fusions de paires |
| **Chat template** | Gabarit qui met en forme les messages avec les tokens spéciaux du modèle |
| **Cross-attention** | Attention entre deux séquences différentes |
| **Decoder-only** | Architecture causale produisant du texte (GPT, Llama) |
| **DPO** | Optimisation directe de préférences, sans modèle de récompense |
| **Embedding** | Représentation d'un token ou texte par un vecteur |
| **FFN** | Réseau feed-forward appliqué à chaque token dans un bloc |
| **FlashAttention** | Attention exacte et économe en mémoire |
| **GQA / MQA** | Têtes d'attention qui partagent leurs K/V pour réduire le KV cache |
| **GRPO** | RL par comparaison au sein d'un groupe de réponses, sans modèle de valeur |
| **KV cache** | Mémoire des clés/valeurs déjà calculées pendant la génération |
| **Logits** | Scores bruts, un par token du vocabulaire, avant le softmax |
| **LoRA** | Adaptation de rang faible : `ΔW = BA` |
| **MoE** | *Mixture of Experts* : plusieurs FFN, un routeur n'en active que quelques-uns |
| **NF4** | Format 4 bits adapté à la distribution normale des poids (QLoRA) |
| **Oubli catastrophique** | Perte de capacités antérieures après un fine-tuning |
| **PEFT** | *Parameter-Efficient Fine-Tuning* : entraîner peu de paramètres |
| **Perplexité** | `exp(perte)` : hésitation moyenne du modèle |
| **Prefill / decode** | Lecture du prompt en parallèle / génération token par token |
| **QLoRA** | LoRA sur modèle quantifié en 4 bits |
| **Quantification** | Représenter les poids avec moins de bits |
| **Rang (`r`)** | Dimension interne de l'adaptateur LoRA |
| **RLHF** | Apprentissage par renforcement à partir de retours humains |

| Terme | Sens |
|---|---|
| **RoPE** | Positions rotatives : encodent la position relative dans `q` et `k` |
| **SFT** | *Supervised Fine-Tuning* : entraînement sur des paires instruction/réponse |
| **Softmax** | Convertit des scores en probabilités qui somment à 1 |
| **Token** | Morceau de texte, unité de base du modèle |
| **Top-k / top-p** | Restrictions d'échantillonnage sur les tokens les plus probables |
| **TTFT** | *Time to first token* : délai avant le premier token généré |
| **Weight tying** | Partage de la matrice d'embedding entre l'entrée et la sortie |

PARTIE B

# Questions d'entretien Transformers, LLM et LoRA

# Entretien Transformers, LLM et LoRA : questions et réponses

Plus de 200 questions classées par thème, avec des réponses **courtes**, comme on les donnerait à l'oral. Le cours complet est dans `formation-transformers-lora.md`, le code dans `labs/mini_transformer_lora.py`. Pour le RAG et le coût des appels, voir `entretien-rag.md`.

**Comment t'entraîner** : cache la réponse, réponds à voix haute en 30 à 60 secondes, compare. Pour les formules et le code, **écris de mémoire** avant de vérifier.

## Sommaire

A. LLM et tokenisation · B. Attention et Transformer · C. Pré-entraînement · D. Inférence et KV cache · E. Mémoire et entraînement distribué · F. Choisir et fine-tuning · G. **LoRA** · H. QLoRA et variantes · I. Données, alignement et évaluation · J. Quantification et déploiement · K. Conception de systèmes · L. Questions pièges · M. Exercices de code · N. Les erreurs à éviter

---

## A. LLM et tokenisation

**1. Qu'est-ce qu'un LLM ?**
Un réseau de neurones (généralement un Transformer decoder-only) qui prédit la **distribution de probabilité du token suivant** à partir des précédents. Répondre, résumer ou coder découle de cette prédiction répétée.

**2. Qu'est-ce que la génération autorégressive ?**
On tire un token, on l'ajoute à l'entrée, on recommence. La probabilité d'une séquence est le produit des probabilités conditionnelles $P(t_i \mid t_{<i})$.

**3. Pourquoi le Transformer a-t-il remplacé les RNN et LSTM ?**
Les RNN lisent séquentiellement (impossible à paralléliser) et oublient le début des longues séquences. Le Transformer relie **directement** tous les tokens par l'attention et calcule tout en **parallèle**, ce qui permet de passer à l'échelle.

**4. Qu'est-ce qu'un token ?**
Un morceau de texte du vocabulaire (mot fréquent, sous-mot, caractère ou octet). Le modèle manipule des identifiants entiers, et limites comme coûts se comptent en tokens.

**5. Pourquoi des sous-mots plutôt que des mots ou des caractères ?**
Les mots entiers donnent un vocabulaire énorme et des mots inconnus ; les caractères donnent des séquences très longues. Les sous-mots sont le compromis : mots fréquents en un token, mots rares découpés, aucun mot inconnu.

**6. Explique BPE.**
On part de petites unités (octets) et on **fusionne itérativement la paire adjacente la plus fréquente** en un nouveau token, jusqu'à la taille de vocabulaire voulue. À l'encodage, on rejoue les fusions dans l'ordre appris.

**7. Quel est l'intérêt du BPE au niveau des octets ?**
L'alphabet de départ est les 256 octets : **tout texte** (accents, emoji, autres alphabets) est encodable, sans jamais de token « inconnu ».

**8. BPE, WordPiece, Unigram : la différence ?**
BPE fusionne la paire la plus **fréquente**. WordPiece choisit les fusions qui maximisent la **vraisemblance** (BERT). Unigram part d'un gros vocabulaire et **retire** les tokens les moins utiles (SentencePiece).

**9. Pourquoi le français ou l'arabe coûtent-ils plus de tokens que l'anglais ?**
Le vocabulaire est appris surtout sur de l'anglais : les autres langues sont découpées en morceaux plus petits. Même texte = plus de tokens = plus cher et moins de contexte utile.

**10. Pourquoi les LLM comptent-ils mal les lettres d'un mot ?**
Ils voient des tokens, pas des lettres. « fraise » peut être un seul token : les lettres ne sont pas directement accessibles.

**11. Pourquoi l'arithmétique est-elle difficile ?**
Les nombres sont découpés de façon irrégulière (« 1234 » en « 12 » et « 34 »), et le modèle apprend des motifs statistiques, pas un algorithme exact.

**12. Qu'est-ce que les tokens spéciaux ?**
Début et fin de séquence, séparateurs de rôles d'un chat, etc. Les oublier ou en utiliser de mauvais dégrade le fine-tuning sans erreur visible (par exemple un modèle qui ne s'arrête jamais faute de token de fin).

**13. Peut-on changer le tokenizer d'un modèle ?**
Pas sans ré-entraîner l'embedding et la sortie : le tokenizer est lié au modèle. Étendre le vocabulaire demande un entraînement supplémentaire.

**14. Qu'est-ce qu'un embedding de tokens ?**
Une ligne d'une matrice `V × d` apprise : chaque identifiant est transformé en un vecteur de dimension `d`.

**15. Qu'est-ce que le *weight tying* ?**
Réutiliser la matrice d'embedding comme matrice de projection de sortie : économise `V × d` paramètres et couple lecture et écriture des tokens.

**16. Que sont les logits ?**
Les scores bruts, un par token du vocabulaire, produits en sortie avant le softmax.

**17. Que fait le softmax ?**
Il convertit des scores en probabilités positives qui somment à 1 : $\mathrm{softmax}(z_i) = \exp(z_i) / \Sigma \exp(z_j)$.

**18. Qu'est-ce que la perplexité ?**
`exp(perte d'entropie croisée)` : le nombre moyen de tokens équiprobables entre lesquels le modèle hésite. Plus bas = mieux.

**19. Quelle perte initiale attendre d'un modèle non entraîné ?**
`ln(V)` (distribution uniforme). Dans mon mini-modèle ( `V = 30` ) : 3,41 pour `ln 30 = 3,40`. Un écart important signale un bug d'initialisation.

**20. Quelle est la différence entre un modèle de base et un modèle « instruct » ?**
Le modèle de base **complète** du texte (pré-entraîné). Le modèle instruct a subi un **SFT** puis un alignement (RLHF/DPO) pour suivre des instructions et dialoguer.

---

## B. Attention et Transformer

**21. Écris la formule de l'attention.**
`Attention(Q, K, V) = softmax(Q Kᵀ / √d_k) V`.

**22. Quel est le rôle de Q, K et V ?**
`Q` (query) : ce que je cherche. `K` (key) : ce que je contiens pour être trouvé. `V` (value) : l'information transmise. Les scores `Q·K` pondèrent la moyenne des `V`.

**23. Pourquoi diviser par `√d_k` ?**
Si `q` et `k` ont des composantes de variance 1, leur produit scalaire a une variance de `d_k`. Sans normalisation, les scores sont grands, le softmax sature et les gradients s'annulent.

**24. Qu'est-ce que le masque causal et pourquoi ?**
Une matrice triangulaire qui met `−∞` sur les positions futures avant le softmax, pour que chaque token ne voie que le passé. Il empêche la fuite du futur et permet d'entraîner **toutes les positions en un seul passage**.

**25. Pourquoi plusieurs têtes d'attention ?**
Chaque tête a ses propres projections et peut se spécialiser (syntaxe, coréférence, position). On découpe `d_model` en `h` têtes de dimension `d_model/h`, puis on concatène et on projette.

**26. Quelle est la complexité de l'attention ?**
`O(T² · d)` en temps et `O(T²)` en mémoire pour la matrice de scores (version naïve) : quadratique en la longueur `T`.

**27. Comment FlashAttention améliore-t-il cela ?**
Il calcule l'attention **exacte** par blocs en gardant les calculs dans la mémoire rapide du GPU, sans écrire la matrice `T × T` en mémoire principale. Mêmes résultats, bien moins de mémoire et plus rapide (le goulot est la bande passante mémoire).

**28. FlashAttention est-il une approximation ?**
Non. C'est le même calcul réorganisé pour réduire les lectures/écritures mémoire.

**29. Qu'est-ce que la self-attention et la cross-attention ?**
Self : `Q`, `K`, `V` viennent de la **même** séquence. Cross : `Q` vient d'une séquence, `K` et `V` d'une autre (encoder-decoder, multimodal).

**30. MHA, MQA, GQA : la différence ?**
MHA : chaque tête a ses `K` / `V`. MQA : **une seule** paire `K` / `V` partagée par toutes les têtes. GQA : les têtes sont réparties en **groupes** qui partagent une paire. Le but est de réduire le KV cache.

**31. Pourquoi GQA plutôt que MQA ?**
GQA est un compromis : presque la qualité de MHA avec presque l'économie mémoire de MQA.

**32. Pourquoi l'attention a-t-elle besoin d'une information de position ?**
Elle est invariante à l'ordre : sans position, « le chien mord l'homme » et « l'homme mord le chien » donnent les mêmes calculs.

**33. Quelles méthodes de position connais-tu ?**
Sinusoïdale (ajoutée), apprise (table), **RoPE** (rotation de `q` et `k`), ALiBi (malus selon la distance).

**34. Explique RoPE.**
On découpe `q` et `k` en paires de dimensions et on fait tourner chaque paire d'un angle `position × fréquence`. Le produit scalaire résultant ne dépend que de l'**écart** de position : la position relative est capturée naturellement.

**35. Comment étendre le contexte d'un modèle RoPE ?**
Interpolation de positions, NTK-aware, YaRN : on compresse les angles pour rester dans la plage vue à l'entraînement, avec un court fine-tuning sur longues séquences.

**36. Que fait une connexion résiduelle ?**
`x + f(x)` : chaque sous-couche ajoute une correction au flux principal. Elle permet aux gradients de traverser de nombreuses couches.

**37. LayerNorm et RMSNorm ?**
LayerNorm soustrait la moyenne et divise par l'écart type. RMSNorm divise seulement par la racine de la moyenne des carrés : plus simple, aussi efficace (Llama).

**38. Pre-norm ou post-norm ?**
Pre-norm (norme avant la sous-couche) est plus **stable** pour les modèles profonds. Post-norm est le Transformer original.

**39. Que fait le feed-forward (FFN) ?**
Appliqué à **chaque token indépendamment** : élargit la dimension, applique une non-linéarité, la réduit. Il porte la majorité des paramètres et une grande part des connaissances factuelles.

**40. Qu'est-ce que SwiGLU ?**
`W_down · (SiLU(W_gate x) ⊙ W_up x)` : une FFN à trois matrices avec un mécanisme de porte, `d_ff ≈ 8/3 d` pour garder le même nombre de paramètres.

**41. Rôle respectif de l'attention et du FFN ?**
L'attention fait **communiquer** les tokens ; le FFN **calcule** sur chaque token. Les deux alternent.

**42. Encoder-only, decoder-only, encoder-decoder ?**
Encoder (BERT) : attention bidirectionnelle, classification/embeddings. Decoder (GPT, Llama) : causal, génération. Encoder-decoder (T5) : traduction, résumé.

**43. Pourquoi le decoder-only domine-t-il ?**
Un seul objectif simple (le token suivant), s'entraîne sur tout texte brut, passe très bien à l'échelle et gère nativement le prompting et le few-shot.

**44. BERT peut-il générer du texte ?**
Mal : il est bidirectionnel et entraîné à remplir des mots masqués, pas à prédire la suite.

**45. Compte les paramètres d'un Transformer.**
Par bloc ≈ `12 d²` (attention `4d²` + FFN `8d²`), total ≈ `12 N d² + V d`. GPT-2 small : `12·12·768² ≈ 85 M` + embeddings `38,6 M` ≈ 124 M.

**46. Et pour Llama-2 7B ?**
Par couche : attention $4\,d^2 = 67\ M$, FFN $3 \cdot d \cdot d_ff = 3 \cdot 4096 \cdot 11008 = 135\ M$, soit 202 M ; ×32 = 6,48 G ; plus 262 M d'embeddings = **6,74 G**.

**47. Où sont la plupart des paramètres d'un bloc ?**
Dans le **FFN** (environ les deux tiers). D'où l'importance de l'adapter avec LoRA.

**48. Comment l'attention scale-t-elle avec la longueur du contexte ?**
Quadratiquement en calcul (sans astuce) ; le KV cache croît linéairement. D'où fenêtre glissante, GQA, FlashAttention et modèles à coût linéaire.

**49. Que signifie « lost in the middle » ?**
Les modèles exploitent moins bien l'information placée au **milieu** d'un long contexte que celle du début ou de la fin.

**50. Pourquoi le Transformer est-il parallélisable à l'entraînement mais pas à la génération ?**
À l'entraînement, tous les tokens sont connus : toutes les positions se calculent en parallèle (masque causal). À la génération, chaque token dépend du précédent : séquentiel.

---

## C. Pré-entraînement

**51. Quel est l'objectif de pré-entraînement d'un LLM ?**
Minimiser l'entropie croisée de la prédiction du token suivant sur un énorme corpus de texte.

**52. Que sont les lois d'échelle ?**
La perte décroît de façon régulière (loi de puissance) avec le nombre de paramètres, de données et de calcul (Kaplan, 2020).

**53. Qu'a montré Chinchilla ?**
Pour un budget de calcul donné, paramètres et tokens doivent croître ensemble : environ **20 tokens par paramètre**. Beaucoup de modèles étaient sous-entraînés.

**54. Pourquoi les modèles récents sont-ils sur-entraînés par rapport à Chinchilla ?**
Le coût d'entraînement est payé une fois, celui d'**inférence** à chaque requête : un modèle plus petit, entraîné sur beaucoup plus de tokens, est moins cher à servir à qualité comparable.

**55. Estime le calcul d'entraînement.**
$\approx 6 \times N \times D$ FLOPs ($N$ paramètres, $D$ tokens). Un 7B sur 2 000 G de tokens : $6 \times 7{\cdot}10^{9} \times 2{\cdot}10^{12} \approx 8{,}4{\cdot}10^{22}$.

**56. Quels sont les FLOPs par token ?**
Passe avant $\approx 2N$, entraînement (avant + arrière) $\approx 6N$.

**57. Quel optimiseur et quel planning de taux d'apprentissage ?**
AdamW ($\beta_2 \approx 0{,}95$, weight decay ≈ 0,1), échauffement linéaire puis décroissance en cosinus, écrêtage de gradient ≈ 1,0.

**58. Pourquoi un échauffement (*warmup*) ?**
Au début, les statistiques d'Adam sont imprécises et les gradients grands : un taux trop haut fait diverger.

**59. Pourquoi écrêter les gradients ?**
Pour limiter les pics de perte (*loss spikes*) causés par quelques mini-batchs aberrants.

**60. Pourquoi la déduplication des données est-elle importante ?**
Les doublons font **mémoriser** le modèle (fuite de données, moins de généralisation) et gaspillent du calcul.

**61. Qu'est-ce que la contamination ?**
La présence dans les données d'entraînement de questions ou réponses d'un benchmark : les scores sont gonflés.

**62. Qu'est-ce que le pré-entraînement continu ?**
Poursuivre le pré-entraînement sur un corpus de domaine ou de langue, pour de grandes adaptations de vocabulaire ou de connaissances, avant un SFT.

**63. Qu'est-ce que le *curriculum* de données ?**
Ordonner ou mélanger les données (par qualité, difficulté, domaine) au cours de l'entraînement.

**64. Que signifie « émergence » ?**
Des capacités qui semblent apparaître brusquement avec la taille. Débat : une partie de l'effet dépend de la métrique (discontinue) plutôt que du modèle.

**65. Pourquoi bf16 plutôt que fp16 ?**
Même étendue d'exposants que fp32 : pas de dépassement, pas besoin de *loss scaling*. fp16 a une plage étroite et exige un facteur d'échelle.

**66. Comment vérifier qu'un entraînement est sain ?**
Perte initiale ≈ `ln V`, perte qui baisse, **sur-apprentissage d'un petit lot** (20 exemples), normes de gradient stables, pas de NaN.

---

## D. Inférence et KV cache

**67. Quelles stratégies de décodage connais-tu ?**
Glouton, température, top-k, top-p (noyau), beam search, pénalité de répétition.

**68. Que fait la température ?**
Divise les logits avant le softmax : `T < 1` rend la distribution plus tranchée, `T > 1` plus plate, `T → 0` équivaut au glouton.

**69. Top-k ou top-p ?**
Top-k garde un nombre **fixe** de candidats. Top-p garde le plus petit ensemble dont la probabilité cumulée dépasse `p` : il **s'adapte** (peu de choix quand le modèle est sûr, plus quand il hésite).

**70. Quand utiliser une température basse ?**
Pour du factuel, de l'extraction, du JSON : 0 à 0,2. Pour la créativité : 0,7 à 1.

**71. Le décodage glouton est-il optimal ?**
Non : choisir le meilleur token à chaque pas ne donne pas la séquence la plus probable globalement. Le beam search explore plusieurs candidats.

**72. Qu'est-ce que le prefill et le decode ?**
Prefill : lecture du prompt **en parallèle** (limité par le calcul), détermine le TTFT. Decode : un token à la fois (limité par la **mémoire**), détermine le débit.

**73. Qu'est-ce que le KV cache ?**
On stocke les `K` et `V` des tokens déjà traités pour ne calculer que ceux du **nouveau** token à chaque pas. Résultats identiques, bien moins de calcul.

**74. Pourquoi le cache est-il valide ?**
Grâce au masque causal : les `K` et `V` d'un token ne dépendent pas des tokens suivants, donc ne changent jamais.

**75. Comment calcules-tu la taille du KV cache ?**
`2 × couches × têtes_KV × dim_tête × octets` par token. Llama-2 7B (32 couches, 32 têtes KV, dim 128, fp16) : `2×32×32×128×2 = 512 Kio` par token, soit **2 Gio pour 4 096 tokens**.

**76. Comment GQA modifie-t-il ce calcul ?**
Il réduit le nombre de têtes KV. Avec 8 têtes au lieu de 32 : cache **divisé par 4** (128 Kio par token).

**77. Pourquoi le KV cache limite-t-il le nombre d'utilisateurs ?**
Il est **par requête** : 50 requêtes de 4 096 tokens sur un 7B sans GQA = 100 Gio de cache.

**78. Pourquoi l'inférence est-elle limitée par la mémoire ?**
À chaque token, on relit tous les poids pour très peu de calcul. Le débit dépend donc de la bande passante mémoire.

**79. Pourquoi la quantification accélère-t-elle la génération ?**
Moins d'octets à lire par token. Comme le decode est limité par la mémoire, c'est un vrai gain de vitesse.

**80. Qu'est-ce que le batching continu ?**
Ajouter et retirer des requêtes du lot **à chaque pas** au lieu d'attendre la plus lente : meilleure utilisation du GPU.

**81. Qu'est-ce que PagedAttention ?**
Le KV cache est découpé en pages, comme la mémoire virtuelle d'un OS : moins de fragmentation et de gaspillage (vLLM).

**82. Qu'est-ce que le décodage spéculatif ?**
Un petit modèle propose plusieurs tokens, le grand les **vérifie en une passe**. Résultat identique, plus rapide quand les propositions sont souvent acceptées.

**83. Qu'est-ce que le TTFT ?**
*Time to first token* : délai avant le premier token. Il dépend de la longueur du prompt (prefill).

**84. Comment réduire la latence perçue ?**
Streaming des tokens, modèle plus petit, prompt plus court, cache de préfixe, quantification.

**85. Comment contraindre une sortie JSON ?**
Décodage contraint par une grammaire (on masque les tokens invalides), mode JSON du fournisseur, puis validation en code.

**86. Qu'est-ce que le cache de préfixe ?**
Réutiliser le KV cache de prompts qui partagent un début (instructions système) : économise le prefill.

**87. Pourquoi la même requête peut-elle donner des réponses différentes avec `T = 0` ?**
Non-déterminisme des calculs en virgule flottante sur GPU (ordre d'accumulation, batching) : de petits écarts changent parfois un `argmax`.

**88. Qu'est-ce que la pénalité de répétition ?**
Réduire le score des tokens déjà générés pour éviter les boucles ; à doser car elle peut dégrader les réponses qui doivent répéter des termes.

---

## E. Mémoire et entraînement distribué

**89. De quoi dépend la mémoire en inférence ?**
Poids ( `paramètres × octets` ) + KV cache + activations (petites) + marge.

**90. Combien de mémoire pour un 7B ?**
Poids : fp32 27 Go, bf16 **13,5 Go**, int8 6,7 Go, int4 **3,4 Go**, sans compter le KV cache.

**91. Pourquoi un fine-tuning complet de 7B demande-t-il environ 108 Go ?**
`16 octets par paramètre` : poids bf16 (2) + gradient (2) + copie maître fp32 (4) + moment `m` (4) + moment `v` (4). Plus les activations.

**92. Quelle mémoire pour un 70B en fine-tuning complet ?**
`70 G × 16 ≈ 1,1 To` : plusieurs GPU et un partage des états (FSDP/ZeRO).

**93. Qu'est-ce que le gradient checkpointing ?**
On ne garde pas toutes les activations : on les **recalcule** pendant la passe arrière. Économise de la mémoire au prix d'environ 30 % de calcul en plus.

**94. Qu'est-ce que l'accumulation de gradients ?**
Cumuler les gradients de plusieurs micro-batchs avant la mise à jour : gros batch effectif avec peu de mémoire.

**95. DDP, FSDP/ZeRO, parallélisme tenseur et pipeline ?**
DDP : chaque GPU a une **copie** du modèle, on partage les données. ZeRO/FSDP : on **partage** états d'optimiseur, gradients et poids entre GPU. Tenseur : on découpe les matrices. Pipeline : on découpe les couches.

**96. Que partitionne ZeRO ?**
Stade 1 : états d'optimiseur. Stade 2 : + gradients. Stade 3 : + poids. Plus on partage, plus on économise de mémoire, plus on communique.

**97. Qu'est-ce qui est limitant : calcul ou communication ?**
Cela dépend de l'échelle : sur de gros clusters, la **communication** entre GPU devient souvent le goulot ; d'où le choix du type de parallélisme selon la topologie.

**98. Pourquoi les optimiseurs 8 bits ?**
Les états d'Adam occupent 8 octets par paramètre : les quantifier réduit fortement la mémoire.

---

## F. Choisir et fine-tuning

**99. Prompting, RAG ou fine-tuning : comment choisir ?**
Du moins au plus coûteux : prompt, RAG, fine-tuning. On monte seulement si l'échelon inférieur ne suffit pas, **prouvé par une évaluation**. RAG pour les **connaissances**, fine-tuning pour le **comportement**.

**100. Pourquoi le fine-tuning n'est-il pas adapté pour ajouter des faits ?**
Il apprend mieux des motifs de style que des faits isolés, peut apprendre à répondre avec assurance sans savoir (hallucination), ne cite pas, et il faut ré-entraîner à chaque changement.

**101. Quand fine-tuner ?**
Format ou style strict, vocabulaire ou raisonnement de domaine, consignes complexes fiables, **réduire le coût** en faisant égaler un gros modèle par un petit sur une tâche étroite.

**102. Peut-on combiner RAG et fine-tuning ?**
Oui : fine-tuner pour mieux **utiliser** un contexte récupéré (citer, refuser, respecter le format) tout en gardant le RAG pour les connaissances.

**103. Que vérifies-tu avant de fine-tuner ?**
Un prompt de référence qui échoue, une évaluation chiffrée, assez de données de qualité, le gain attendu face au coût, les licences.

**104. Qu'est-ce que le SFT ?**
Entraînement supervisé sur des paires (instruction, réponse idéale), avec perte sur la réponse : il apprend le format et le comportement.

**105. Qu'est-ce que l'oubli catastrophique ?**
La perte de capacités générales après un fine-tuning. Parades : taux d'apprentissage et rang modestes, peu d'époques, données mélangées, évaluation de contrôle.

**106. Fine-tuning complet : avantages et inconvénients ?**
Capacité maximale, mais mémoire énorme, une copie complète par tâche, risque d'oubli et de sur-apprentissage.

**107. Qu'est-ce que PEFT ?**
Entraîner un **petit nombre** de paramètres en gardant le modèle gelé : mémoire réduite, adaptateurs minuscules et échangeables.

**108. Quelles familles de PEFT connais-tu ?**
Adapters (modules insérés), prompts doux et prefix tuning (vecteurs virtuels), BitFit (biais), $IA^3$ (vecteurs multiplicatifs), **LoRA** (mise à jour de rang faible).

**109. Pourquoi LoRA a-t-il gagné ?**
Qualité proche du fine-tuning complet, **aucune latence ajoutée** après fusion, adaptateurs minuscules, implémentation simple, compatible avec la quantification (QLoRA).

**110. Inconvénient des adapters classiques ?**
Les couches insérées **ajoutent de la latence** à l'inférence, contrairement à LoRA fusionné.

**111. Inconvénient du prompt tuning ?**
Il consomme du contexte, est moins stable et souvent moins performant, surtout sur les petits modèles.

**112. Combien d'exemples faut-il ?**
Style/format : 100 à 1 000. Tâche étroite : 500 à 5 000. Instructions variées : 1 000 à 50 000. **Qualité avant quantité** : on commence petit et on ajoute là où le modèle échoue.

---

## G. LoRA

**113. Explique LoRA en 30 secondes.**
On **gèle** le modèle de base et on apprend, pour chaque matrice ciblée, une mise à jour de **rang faible** `ΔW = B·A` avec `h = W₀x + (α/r)·B·A·x`. Très peu de paramètres à entraîner, adaptateur de quelques dizaines de Mo, et on peut **fusionner** `W ← W + (α/r)BA` pour n'avoir aucune latence ajoutée.

**114. Quelle est l'hypothèse derrière LoRA ?**
La mise à jour des poids lors d'un fine-tuning a un **rang intrinsèque faible** : on peut l'approcher par un produit de deux petites matrices (dimension intrinsèque faible des tâches d'adaptation).

**115. Donne les dimensions de A et B.**
`W₀ ∈ ℝ^(d_out × d_in)`, `A ∈ ℝ^(r × d_in)`, `B ∈ ℝ^(d_out × r)`, avec `r ≪ min(d_in, d_out)`.

**116. Combien de paramètres LoRA ajoute-t-il par matrice ?**
`r × (d_in + d_out)`, au lieu de `d_in × d_out`. Exemple : `r = 8`, matrice 4096×4096 : 65 536 au lieu de 16,8 M (0,4 %).

**117. Combien de paramètres pour un 7B ?**
`r = 8` sur `q` et `v` : **4,19 M** (0,06 %). `r = 16` sur toutes les couches linéaires : **40 M** (0,59 %). `r = 64` : 160 M (2,4 %).

**118. Quelle taille a le fichier d'adaptateur ?**
`r = 16`, toutes couches, 7B, en bf16 : environ **80 Mo**, contre 13,5 Go pour le modèle.

**119. Pourquoi B est-il initialisé à zéro ?**
Pour que `ΔW = 0` au départ : le modèle démarre **exactement** comme le modèle de base.

**120. Pourquoi A n'est-il pas aussi initialisé à zéro ?**
Si les deux étaient nuls, les gradients de `A` et `B` seraient nuls (chacun dépend de l'autre) et rien n'apprendrait. `A` aléatoire permet à `B` de recevoir des gradients dès le premier pas.

**121. Et si B et A étaient tous deux aléatoires ?**
Le modèle serait perturbé dès le départ ( `ΔW ≠ 0` ) : on perd la garantie de partir du modèle d'origine.

**122. Montre que B apprend avant A.**
Avec `s = α/r` et `g = ∂L/∂h` : `∂L/∂B = s·g·(Ax)ᵀ` et `∂L/∂A = s·Bᵀ·g·xᵀ`. À `B = 0`, le gradient de `A` est nul mais celui de `B` ne l'est pas. Après un pas, `B ≠ 0` et `A` reçoit des gradients.

**123. À quoi sert α ?**
Facteur d'échelle de la mise à jour ( `α/r` ). Il permet de **changer** `r` **sans re-régler le taux d'apprentissage**. Pratique courante : `α = r` ou `2r`.

**124. α augmente-t-il la capacité ?**
Non. Il règle l'**amplitude** de l'adaptation, pas le nombre de degrés de liberté (c'est `r` ).

**125. Qu'est-ce que rsLoRA ?**
Utiliser l'échelle `α/√r` au lieu de `α/r`, plus stable pour de **grands rangs**.

**126. Comment choisir le rang ?**
Commencer à 16 ; monter (32 à 128) si le modèle sous-apprend sur une tâche difficile ; descendre à 4-8 pour du style simple. Un rang plus grand n'est pas toujours mieux : le gain stagne selon la tâche.

**127. Sur quelles matrices appliquer LoRA ?**
Le papier original : `q` et `v`. En pratique : **toutes les couches linéaires** (`q, k, v, o, gate, up, down`) ; QLoRA a montré que c'est important pour égaler le fine-tuning complet. PEFT : `target_modules="all-linear"`.

**128. Pourquoi inclure les couches du FFN ?**
Le FFN porte la majorité des paramètres du bloc ; l'omettre limite la capacité d'adaptation.

**129. Quel taux d'apprentissage pour LoRA ?**
Environ `1e-4` à `2e-4`, bien plus haut que pour un fine-tuning complet (≈ `1e-5`), car on part de `B = 0` et on entraîne peu de paramètres.

**130. Quels autres hyperparamètres ?**
`lora_dropout` (0 à 0,1), 1 à 3 époques, batch effectif 16 à 64, échauffement ≈ 3 % puis cosinus.

**131. LoRA réduit-il le temps de calcul ?**
Peu. Il économise surtout la **mémoire** (gradients et états d'Adam des poids gelés). La passe arrière doit toujours traverser les poids gelés pour propager les gradients aux activations.

**132. Quelle mémoire LoRA économise-t-il exactement ?**
Gradients et états d'optimiseur des poids gelés. Pour un 7B, `r = 16`, toutes couches : adaptateurs ≈ 80 Mo, gradients ≈ 80 Mo, états Adam ≈ 320 Mo : moins de 0,5 Go au lieu des ~108 Go d'un entraînement complet.

**133. Que reste-t-il comme coût mémoire ?**
Les poids du modèle (13,5 Go en bf16) et les **activations**, réduites par le gradient checkpointing.

**134. Comment fusionner un adaptateur ?**
`W ← W + (α/r)·B·A`. Plus de chemin LoRA à l'inférence : aucune latence ajoutée. La fusion est exacte, et inversible tant qu'on conserve `A` et `B`.

**135. Fusionner ou garder séparé ?**
Fusionné : un seul usage, aucune surcharge. Séparé : **plusieurs adaptateurs** sur un même modèle de base, changés par requête (service multi-adaptateurs).

**136. Comment servir 100 clients avec 100 fine-tunings ?**
Un seul modèle de base en mémoire et 100 adaptateurs LoRA (quelques dizaines de Mo chacun), sélectionnés par requête avec un serveur multi-adaptateurs. Bien moins cher que 100 modèles complets.

**137. Peut-on combiner plusieurs adaptateurs ?**
Oui : somme pondérée, ou méthodes de fusion (TIES, DARE) ; le résultat est empirique, à évaluer.

**138. Quelles sont les limites de LoRA ?**
« Apprend moins et oublie moins » : sur des tâches qui demandent beaucoup de nouvelles capacités (code, maths à grande échelle), il peut rester derrière le fine-tuning complet. Sensible aux hyperparamètres ; n'ajoute pas bien de connaissances factuelles.

**139. LoRA oublie-t-il moins que le fine-tuning complet ?**
Généralement oui : il modifie le modèle dans un sous-espace restreint, ce qui préserve davantage les capacités d'origine.

**140. Le rang de `BA` peut-il dépasser `r` ?**
Non : `rang(BA) ≤ r`. Les mises à jour sont contraintes à un sous-espace de rang au plus `r`.

**141. LoRA modifie-t-il la latence d'inférence ?**
Fusionné : aucune. Non fusionné : légère surcharge (deux petits produits matriciels en plus).

**142. Que prouve ton lab LoRA ?**
(1) Sortie identique à la base au départ. (2) Paramètres entraînés = `r(d_in + d_out)` par matrice. (3) Un modèle pré-entraîné sur `n, n+1...` apprend `n, n+2...` avec 3 072 paramètres sur 120 512, perte **5,97** → **0,02**. (4) Seuls `A` et `B` bougent. (5) La fusion est exacte.

**143. Que se passe-t-il si `lora_dropout` est trop haut ?**
Le chemin LoRA est trop bruité, l'apprentissage ralentit. Il aide surtout avec de petits jeux de données.

**144. LoRA est-il utilisable pour autre chose que le texte ?**
Oui : modèles de diffusion (styles d'image), modèles de vision, multimodaux, encoders (embeddings, reranker).

**145. Peut-on fine-tuner les embeddings d'entrée avec LoRA ?**
On peut l'appliquer à la couche d'embedding, mais c'est moins courant ; si on ajoute des tokens spéciaux, il faut entraîner (ou « sauvegarder ») les couches d'embedding et de sortie concernées.

**146. Comment LoRA se compare-t-il aux adapters de Houlsby ?**
Même esprit (petits modules entraînés), mais LoRA est **additif sur les poids** et fusionnable : zéro latence supplémentaire.

**147. Comment choisir entre LoRA et fine-tuning complet ?**
Je commence par LoRA (peu de mémoire, rapide, réversible). Je passe au complet si l'évaluation montre une limite de capacité (grand volume de données, nouvelle compétence difficile) et si j'ai le matériel.

**148. Pourquoi le rang suffit-il souvent à `8` ou `16` ?**
Pour des tâches de style, de format ou de consigne, la mise à jour utile est de rang faible. Les tâches plus riches demandent plus de capacité.

**149. Comment déboguer un LoRA qui n'apprend pas ?**
Vérifier que les paramètres LoRA sont bien `requires_grad`, que `B` bouge, le taux d'apprentissage, les modules ciblés, le masquage des labels, puis tenter de **sur-apprendre 20 exemples**.

**150. Que sauvegarde-t-on après l'entraînement ?**
Seulement l'adaptateur (`A`, `B` et la configuration : `r`, `α`, modules ciblés) ; on le recharge sur le **même** modèle de base.

**151. Pourquoi faut-il le même modèle de base pour recharger un adaptateur ?**
`ΔW` est défini **relativement** à $W_0$ : appliqué à d'autres poids, il n'a pas de sens.

---

## H. QLoRA et variantes

**152. Qu'est-ce que QLoRA ?**
LoRA appliqué à un modèle de base **quantifié en 4 bits** (NF4) et gelé, avec des adaptateurs en 16 bits. Les gradients traversent les poids 4 bits. Il permet de fine-tuner un 65B sur un GPU de 48 Go.

**153. Quelles sont les trois innovations de QLoRA ?**
**NF4** (format 4 bits adapté aux poids normalement distribués), **double quantification** (on quantifie les constantes de quantification), **optimiseurs paginés** (états d'Adam débordant vers la RAM CPU pour éviter l'*out of memory*).

**154. Qu'est-ce que NF4 ?**
*4-bit NormalFloat* : 16 niveaux placés selon les quantiles d'une loi normale, donc plus précis qu'un int4 uniforme pour des poids de distribution normale.

**155. Pourquoi la double quantification ?**
Chaque bloc de poids a une constante de quantification stockée en 32 bits ; la quantifier à son tour économise environ 0,4 bit par paramètre.

**156. Comment les calculs se font-ils avec des poids 4 bits ?**
Chaque bloc est **déquantifié à la volée en bf16** pour le calcul, puis jeté. Les adaptateurs et leurs gradients restent en 16 bits.

**157. QLoRA est-il plus lent que LoRA ?**
Oui : la déquantification à la volée ajoute du temps. On échange **vitesse contre mémoire**.

**158. Combien de mémoire pour un 7B en QLoRA ?**
Poids NF4 ≈ 3,4 Go, adaptateurs + gradients + états < 0,5 Go, plus les activations (dépendent du batch et de la longueur). Un GPU de 12 à 16 Go suffit souvent.

**159. QLoRA dégrade-t-il la qualité ?**
Le papier rapporte une qualité équivalente au fine-tuning 16 bits sur les benchmarks étudiés. Sur ta tâche : **mesure**.

**160. Peut-on fusionner l'adaptateur dans le modèle 4 bits ?**
Mal : fusionner une mise à jour fine dans des poids déjà arrondis perd de la précision. On recharge le modèle en 16 bits, on fusionne, puis on **re-quantifie** si besoin.

**161. Qu'est-ce que DoRA ?**
Il décompose le poids en **magnitude** et **direction** : LoRA adapte la direction, la magnitude est apprise séparément. Souvent meilleur à faible rang, un peu plus coûteux.

**162. Qu'est-ce que LoRA+ ?**
Deux taux d'apprentissage : plus grand pour `B` que pour `A`. Apprentissage plus rapide et meilleur.

**163. Qu'est-ce qu'AdaLoRA ?**
Il **alloue le rang** dynamiquement entre les matrices selon leur importance, à budget total fixé.

**164. Qu'est-ce que VeRA ?**
`A` et `B` sont aléatoires, **partagés et gelés** ; on n'apprend que de petits vecteurs d'échelle : encore moins de paramètres que LoRA.

**165. Qu'est-ce que LoftQ ?**
Une initialisation de LoRA qui **compense l'erreur de quantification** du modèle 4 bits : meilleur départ pour QLoRA.

**166. Qu'est-ce que PiSSA ?**
Initialise l'adaptateur avec les composantes **principales** (SVD) du poids et gèle le résidu : convergence plus rapide.

**167. Comment choisir une variante ?**
Je commence par LoRA/QLoRA standard sur toutes les couches linéaires. Je teste une variante seulement si l'évaluation montre une limite, **sur la même évaluation**.

---

# I. Données, alignement et évaluation

**168. Qu'a montré LIMA ?**
Qu'un bon modèle de base affiné sur **1 000 exemples soignés** peut répondre très bien : le pré-entraînement apporte les connaissances, le fine-tuning apprend surtout le **format et le style**.

**169. Qualité ou quantité des données ?**
**Qualité** d'abord : 500 exemples excellents valent mieux que 50 000 bruités. On étend ensuite là où le modèle échoue.

**170. Que contient une bonne checklist de données ?**
Cohérence, exactitude, diversité, représentativité, déduplication, équilibre, anonymisation, absence de contamination, jeu de validation séparé, versionnement.

**171. D'où viennent les données ?**
Données réelles nettoyées, annotation humaine, **données synthétiques** (modèle plus fort), et correction des échecs du modèle actuel (souvent le meilleur rendement).

**172. Risques des données synthétiques ?**
Erreurs propagées, perte de diversité, et **conditions d'utilisation** du modèle générateur (certaines interdisent d'entraîner un concurrent). Filtrer et relire un échantillon.

**173. Que sont le *chat template* et pourquoi est-il critique ?**
Le gabarit qui met les messages en forme avec les tokens spéciaux du modèle. Fine-tuner avec un gabarit et inférer avec un autre dégrade silencieusement le comportement.

**174. Pourquoi masquer le prompt dans la perte ?**
Pour apprendre à **produire la réponse**, pas à recopier la question : labels du prompt à `-100`, perte uniquement sur les tokens de l'assistant.

**175. Qu'est-ce que le *packing* ?**
Concaténer plusieurs exemples courts dans une même séquence pour remplir le contexte et accélérer l'entraînement ; attention aux fuites d'attention entre exemples.

**176. Pourquoi inclure des exemples de refus ?**
Sinon le modèle apprend à **toujours répondre**, même sans information, ce qui augmente les hallucinations.

**177. Que se passe-t-il si le token de fin manque dans les données ?**
Le modèle ne sait pas s'arrêter et génère jusqu'à la limite.

**178. Explique RLHF.**
SFT, puis entraînement d'un **modèle de récompense** sur des préférences humaines (perte de Bradley-Terry), puis optimisation de la politique par renforcement (souvent PPO) avec une **pénalité KL** vers le modèle de référence.

**179. Quels sont les problèmes de RLHF ?**
Complexe (politique, référence, récompense, valeur), instable, et **détournement de la récompense** (le modèle exploite les failles du juge, par exemple des réponses longues et flatteuses).

**180. Explique DPO.**
On entraîne directement sur des paires (préférée, rejetée), sans modèle de récompense ni renforcement : `L = -log σ(β[(log π(y_w) - log π_ref(y_w)) - (log π(y_l) - log π_ref(y_l))])` . On augmente la probabilité de la préférée **relativement à la référence**.

**181. Que vaut la perte DPO quand la politique égale la référence ?**
`ln 2 ≈ 0,693` : aucune préférence apprise (vérifié dans le lab).

**182. Que fait β dans DPO ?**
Il règle la force du maintien près de la référence : petit β = on s'autorise à s'éloigner davantage. Valeur courante : 0,1.

**183. Comment DPO se combine-t-il avec LoRA ?**
La référence s'obtient **sans copie du modèle** en désactivant les adaptateurs. DPO + LoRA tient sur un seul GPU.

**184. Qu'est-ce que ORPO, KTO, SimPO ?**
ORPO : SFT et préférence en une étape, sans référence. KTO : besoin d'un simple « bon/mauvais » par réponse, pas de paires. SimPO : sans référence, avec la log-probabilité moyenne comme récompense.

**185. Qu'est-ce que RLAIF / Constitutional AI ?**
Les préférences viennent d'un **modèle** guidé par des principes écrits, au lieu d'humains.

**186. Qu'est-ce que GRPO ?**
Pour chaque prompt, on génère un **groupe** de réponses, on calcule leur récompense, et l'avantage de chacune est `(r - moyenne du groupe) / écart type du groupe` . **Pas de modèle de valeur** : plus léger que PPO. Utilisé pour les modèles de raisonnement.

**187. Qu'est-ce que le RLVR ?**
RL avec **récompenses vérifiables** (réponse mathématique correcte, tests de code qui passent), sans modèle de récompense appris.

**188. Comment évalues-tu un modèle fine-tuné ?**
Contre **deux références** : le modèle de base avec un bon prompt, et ce même modèle avec RAG si pertinent. Métriques de tâche sur un jeu de test séparé, benchmarks de contrôle (oubli), sécurité, format, coût et latence.

**189. Pourquoi comparer à un bon prompt et pas à un prompt simple ?**
Un fine-tuning qui ne bat pas un prompt bien fait ne vaut pas son coût de maintenance.

**190. Pourquoi la perte de validation ne suffit-elle pas ?**
Elle mesure la vraisemblance des tokens, pas l'**utilité** des réponses ni leur exactitude.

**191. Quels sont les biais d'un LLM-as-judge ?**
Préfère les réponses longues, son propre style, la première position. Parades : critères précis, échelle simple, juge différent du générateur, **calibrage sur notes humaines**.

**192. Comment détecter l'oubli catastrophique ?**
Rejouer quelques benchmarks généraux et des tests de capacités avant/après fine-tuning.

**193. Le fine-tuning peut-il affaiblir la sécurité ?**
Oui, même sur des données inoffensives : retester les refus et mélanger des exemples de sécurité.

**194. Pourquoi 10 exemples de test ne suffisent-ils pas ?**
Trop de variance pour distinguer deux modèles. Il faut des dizaines à des centaines d'exemples variés, et vérifier avec plusieurs graines.

**195. Pourquoi ne pas choisir le meilleur checkpoint sur le jeu de test ?**
Le jeu de test devient alors un jeu d'entraînement et les scores sont optimistes. On choisit sur la validation et on teste **une fois**.

**196. Qu'est-ce que la contamination d'un benchmark ?**
Des exemples du benchmark présents dans les données d'entraînement : les scores ne reflètent plus la généralisation.

---

## J. Quantification et déploiement

**197. Qu'est-ce que la quantification ?**
Représenter les poids avec moins de bits (int8, int4) pour économiser mémoire et accélérer la génération.

**198. PTQ ou QAT ?**
PTQ : on quantifie un modèle déjà entraîné avec un petit jeu de calibration (simple). QAT : on simule la quantification **pendant** l'entraînement (meilleure qualité, plus coûteux).

**199. GPTQ, AWQ, GGUF, NF4 ?**
GPTQ : PTQ 4 bits (information de second ordre), GPU. AWQ : PTQ 4 bits qui protège les poids importants selon les **activations**. GGUF : format de llama.cpp pour CPU et local ( `Q4_K_M` ...). NF4 : 4 bits pour **fine-tuning** (QLoRA).

**200. Quelle perte attendre ?**
8 bits : quasi nulle. 4 bits : modérée, acceptable pour beaucoup d'usages. En dessous : nette. **Mesurer sur la tâche.**

**201. Comment déployer un modèle fine-tuné avec LoRA ?**
Option A : fusionner en bf16, éventuellement re-quantifier, servir un modèle unique. Option B : garder l'adaptateur séparé sur un serveur multi-adaptateurs (un modèle de base, N adaptateurs).

**202. Quels serveurs d'inférence connais-tu ?**
vLLM (batching continu, PagedAttention, multi-LoRA), TGI, llama.cpp/Ollama (local), TensorRT-LLM (très performant sur GPU NVIDIA).

**203. Comment dimensionnes-tu la mémoire GPU d'un service ?**
`poids + KV cache × requêtes simultanées + marge`. 7B bf16 (13,5 Go) + 32 requêtes de 4 096 tokens (32 × 2 Gio) ≈ 80 Go ; avec GQA (cache ÷4) ≈ 30 Go.

**204. API ou modèle auto-hébergé : comment décider ?**
Coût auto-hébergé = GPU loué ÷ débit. Un petit modèle fine-tuné et quantifié peut coûter bien moins par requête **si le volume amortit le GPU**. À faible volume, l'API est souvent moins chère et plus simple.

**205. Qu'est-ce que la fusion de modèles ?**
Combiner plusieurs modèles ou adaptateurs sans entraînement (moyenne de poids, SLERP, TIES, DARE). Empirique : à évaluer.

**206. Que surveilles-tu en production ?**
Latence (TTFT, débit), erreurs, utilisation GPU et mémoire, coût par requête, qualité (échantillons relus, retours utilisateurs), dérive des entrées.

**207. Comment mettre à jour un modèle sans interruption ?**
Déploiement progressif (canary / blue-green), évaluation avant bascule, retour arrière possible vers l'ancienne version (adaptateur ou modèle fusionné versionné).

**208. Comment versionner un fine-tuning ?**
Modèle de base (identifiant exact), version du jeu de données, configuration LoRA, hyperparamètres, graine, scores d'évaluation, adaptateur résultant.

**209. Qu'est-ce qu'un MoE ?**
Plusieurs FFN « experts » et un **routeur** qui en active quelques-uns par token. Paramètres totaux élevés, **actifs** faibles (Mixtral 8x7B : ≈ 47 G au total, ≈ 13 G actifs). La mémoire reste celle de tous les experts.

**210. Qu'est-ce qu'un modèle de raisonnement et ses conséquences d'ingénierie ?**
Entraîné (souvent par RL) à produire une longue chaîne de raisonnement avant la réponse. Les tokens de raisonnement sont **facturés** et allongent la latence ; avec un plafond de tokens trop bas, la réponse peut être vide.

---

## K. Conception de systèmes

Méthode : **clarifier** (volumes, données, contraintes) → **options** → **choix justifié par la mesure** → **risques** → **comment mesurer**.

### Scénario 1 : « Notre assistant interne doit connaître nos documents. Fine-tuner un LLM ? »

- Clarifier : les documents changent-ils ? faut-il citer ? droits d'accès ?
- Recommandation : **RAG** (connaissances changeantes, citations, droits). Fine-tuning éventuel **ultérieur** pour le format et le comportement.
- Mesure : jeu d'évaluation de questions réelles ; comparer prompt, RAG, RAG + modèle fine-tuné.

### Scénario 2 : « Réduire le coût d'une tâche d'extraction JSON à fort volume »

- Baseline : gros modèle + prompt long (coût et latence mesurés).

- Option : **QLoRA sur un petit modèle** avec 500 à 5 000 exemples validés ; sortie contrainte et validation en code.
- Mesure : exactitude, JSON valide, coût par requête, latence. Conclure seulement si le gain de coût amortit données, entraînement et maintenance.

### Scénario 3 : « Un modèle par client (50 clients) »

- Un modèle de base partagé et **un adaptateur LoRA par client**, servi en multi-adaptateurs (vLLM).
- Isolation : l'adaptateur est choisi **côté serveur** selon l'identité authentifiée, jamais par le client.
- Risques : fuite entre clients (tests d'isolation), version de base commune à mettre à jour avec tous les adaptateurs.

### Scénario 4 : « Fine-tuner un 70B alors qu'on n'a qu'un GPU de 80 Go »

- Full fine-tuning impossible (≈ 1,1 To). **QLoRA** : poids NF4 ≈ 35 Go + adaptateurs et activations : possible, avec séquences et batch modestes et gradient checkpointing.
- Alternative : un modèle plus petit (8-14 G) fine-tuné sur le même jeu : souvent suffisant et bien moins cher à servir.

### Scénario 5 : « Le modèle fine-tuné est bon sur notre tâche mais a perdu en sécurité et en général »

- Diagnostic : évaluer refus et benchmarks généraux avant/après.
- Remèdes : rang et taux d'apprentissage plus faibles, moins d'époques, **mélanger** des données générales et de sécurité, ou fusionner partiellement l'adaptateur (échelle < 1).

### Scénario 6 : « Servir un LLM avec 1 000 requêtes simultanées »

- Goulot : KV cache. Leviers : GQA, quantification du modèle et du cache, PagedAttention, batching continu, plusieurs répliques, cache de préfixe.
- Mesure : TTFT p95, débit, coût par million de tokens.

---

## L. Questions pièges

**LoRA entraîne-t-il le modèle moins bien parce qu'il a moins de paramètres ?**
Pas nécessairement : pour du style, du format ou des consignes, la mise à jour utile est de rang faible. Il peut rester derrière sur des tâches très exigeantes en nouvelles capacités. Se prouve par l'évaluation.

**Un rang 256 est-il toujours meilleur qu'un rang 8 ?**
Non : le gain stagne, le risque de sur-apprentissage augmente et la mémoire aussi. On monte seulement si l'évaluation le justifie.

**Augmenter α revient-il à augmenter le rang ?**
Non. α règle l'**amplitude** de la mise à jour ; le rang règle la **capacité**.

**Pourquoi le fine-tuning de mon modèle hallucine-t-il plus ?**
Il a peut-être appris à répondre avec assurance sur des faits qu'il ne maîtrise pas, faute d'exemples de refus. Ajouter des cas « je ne sais pas » et utiliser le RAG pour les faits.

**Pourquoi la perte d'entraînement est-elle presque nulle mais le modèle mauvais en production ?**
Sur-apprentissage : il a mémorisé. Données trop peu nombreuses ou trop d'époques ; il faut une validation séparée et plus de diversité.

**Le KV cache change-t-il la sortie ?**
Non, seulement la vitesse (vérifié dans le lab : mêmes tokens avec et sans cache).

**Pourquoi un modèle 4 bits ne peut-il pas être entraîné normalement ?**
Les poids quantifiés ne sont pas différentiables comme des flottants. Avec QLoRA, on gèle les poids 4 bits et on entraîne des adaptateurs 16 bits à côté.

**Un modèle plus gros est-il toujours meilleur ?**
Non : coût, latence, et un petit modèle bien adapté peut égaler un gros sur une tâche étroite. On décide par la mesure.

**Le contexte long remplace-t-il le RAG ?**
Pas toujours : coût quadratique, information « perdue au milieu », pas de citation ni de droits. Voir la formation RAG.

**LoRA ajoute-t-il de la latence ?**
Non si on fusionne ; légèrement sinon.

**Peut-on fine-tuner sur des données confidentielles sans risque ?**
Non : le modèle peut **mémoriser et restituer** des passages. Anonymiser, dédupliquer, limiter l'accès au modèle et à l'adaptateur.

---

# M. Exercices de code

Les extraits ont tous été exécutés dans `labs/mini_transformer_lora.py`. **Apprends à les écrire sans les relire.**

## Attention causale (avec KV cache)

Voir `CausalSelfAttention` dans le lab. Points qu'on te demandera d'expliquer : la forme des tenseurs `(B, h, T, d_head)`, le masque `tril(diagonal=S - T)` quand il y a un cache, le `softmax` par ligne, la concaténation des têtes puis `W_O`.

## Top-p (noyau)

```
def top_p_filter(logits, p):
    sorted_logits, order = torch.sort(logits, descending=True)
    probs = sorted_logits.softmax(-1)
    remove = probs.cumsum(-1) - probs > p      # on garde le token qui fait franchir p
```

```
    sorted_logits = sorted_logits.masked_fill(remove, float("-inf"))
    return torch.full_like(logits, float("-inf")).scatter(0, order, sorted_logits)
```

## LoRA from scratch

```
class LoRALinear(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base = base
        for p in base.parameters():
            p.requires_grad_(False)
        self.scale = alpha / r
        self.A = nn.Parameter(torch.empty(r, base.in_features))
        self.B = nn.Parameter(torch.zeros(base.out_features, r))
        nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))

    def forward(self, x):
        return self.base(x) + self.scale * (x @ self.A.T @ self.B.T)

    @torch.no_grad()
    def merge(self):
        self.base.weight += self.scale * (self.B @ self.A)
```

Tests à écrire : sortie identique au départ ; seuls `A` et `B` ont `requires_grad` ; nombre de paramètres `= r (d_in + d_out)` ; sortie identique avant et après `merge()`.

## Taille du KV cache

```
def kv_cache_bytes_per_token(n_layers, n_kv_heads, head_dim, bytes_per_value=2):
    return 2 * n_layers * n_kv_heads * head_dim * bytes_per_value

# Llama-2 7B : 524_288 octets par token ; 2 Gio pour 4096 tokens ; ÷4 avec 8 têtes KV (vérifié)
```

## Paramètres LoRA

```
def lora_params(d_in, d_out, r):
    return r * (d_in + d_out)

# 7B, r=8 sur q et v : 2 * 32 * lora_params(4096, 4096, 8) == 4_194_304 (vérifié)
```

## Perte DPO

```
def dpo_loss(policy_chosen, policy_rejected, ref_chosen, ref_rejected, beta=0.1):
    margin = (policy_chosen - ref_chosen) - (policy_rejected - ref_rejected)
```

```
    return -F.logsigmoid(beta * margin).mean()     # = ln 2 quand politique == référence
```

### BPE

Voir `train_bpe`, `bpe_encode`, `bpe_decode` dans le lab (module 2 de la formation).

### RoPE

Voir `rope` dans le lab ; vérifie que `rope(q, m) · rope(k, n)` ne dépend que de `m − n`.

### Questions de code fréquentes

- Écrire l'attention multi-têtes **sans regarder**.
- Implémenter LoRA et le **tester** (sortie initiale identique, fusion exacte).
- Calculer la mémoire d'inférence et d'entraînement d'un modèle donné.
- Écrire un `Dataset` qui **masque le prompt** dans les labels ( `-100` ).
- Lire une courbe de perte et diagnostiquer (sur-apprentissage, divergence, plateau).

---

## N. Les erreurs à éviter

- **Dire que LoRA « ajoute des connaissances ».** Il adapte un comportement ; les faits, c'est le RAG.
- **Promettre qu'un fine-tuning bat toujours le prompting.** Montre-le par une évaluation.
- **Confondre α et r.** α règle l'amplitude, r la capacité.
- **Oublier B = 0** quand on te demande l'initialisation, ou ne pas savoir pourquoi A n'est pas nul.
- **Citer des chiffres non vérifiés** comme des lois. Distingue « calculé » (formule) de « repère » de la pratique.
- **Ignorer la mémoire des activations** en parlant de LoRA : il n'économise que gradients et états d'optimiseur.
- **Fusionner dans un modèle 4 bits** sans mentionner la perte de précision.
- **Parler d'une expérience que tu n'as pas faite.** Si tu n'as pas fine-tuné sur GPU, dis que tu as implémenté et testé LoRA à la main et que tu connais le pipeline Hugging Face.
- **Négliger l'évaluation, les données et la sécurité** pour ne parler que d'architecture.